hochschule burgenland
University of Applied Sciences

Arbeitstitel: Einsatz von LLMs beim Programmieren-Lernen

Masterarbeit

zur Erlangung des akademischen Grades

Master of Science in Engineering

Master Program Cloud Computing Engineering
Department Informationstechnologie

Eingereicht von:	Tobias Wilhelm, BEd
Personenkennzeichen:	2410781008
Datum:	25. Oktober 2026
Betreut von:	Dipl.-Ing. Dr.techn. Georg Macher, BSc MBA

Vorwort

Hier ist der Platz für persönliche Worte wie zum Beispiel Dankesworte an Betreuer, Berater, Freunde und Familie.

Vorname Familienname

Eisenstadt, 27. Juli 2024

Inhaltsverzeichnis

Kurzfassung . . . . . **v**

Abstract . . . . . **vii**

1 Einleitung **1**
1.1 Problemstellung 1
1.2 Wissenschaftliche Fragestellung 1
1.3 Methodischer Ansatz 2
1.4 Aufbau der Arbeit 2

2 Stand des Wissens / Stand der Technik **3**
2.1 Herausforderungen beim Programmierenlernen im tertiären Bildungsbereich 3
2.2 Die Verwendung von LLMs in der Programmierausbildung 3
2.3 Generative KI in Prüfungen und Bewertungsszenarien 4
2.4 LLM-unterstütztes Programmierenlernen 4
2.5 User-Experience-Messungen von LLM-Chatbots mittels User Experience Questionnaires (UEQ) 5
2.6 Zusammenfassung 6

3 Forschungsmethodik **7**
3.1 Motivation & Zielsetzung 7
3.2 Studiendesign und Versuchsaufbau 7
3.3 Erläuterung der Erhebungsinstrumente: persönlicher Fragebogen und UEQ 7
3.4 Konzeption der Programmieraufgabe: Der „Garden-Simulator“ . . 7
3.4.1 Der „Garden-Simulator“ 7
3.4.2 README.md als schriftliche Unterstützung 16
3.4.3 Erstellung der Aufgaben 17
3.4.4 Musterlösung der Aufgaben und Programmierkonzepte . . 18
3.4.5 Lokale Tests zur Selbstüberprüfung 22
3.5 Bewertungsraster zur Evaluierung der Programmieraufgabe 23
3.5.1 Ergänzung der Dimension „Funktionalität und Robustheit“ 25
3.6 Planung der Pilotstudie 25
3.7 Diskussion 25
3.8 Zusammenfassung 25

4 Evaluierung der Studie **27**
4.1 Pilotphase der Studie 27
4.1.1 Durchführung und Erkenntnisse der Pilotstudie 27
4.1.2 Adaptionen für die Hauptstudie 28
4.2 Hauptstudie 28
4.2.1 Evaluierung 28
4.3 Auswertung Fragebogen 1 und UEQ (Fragebogen 2) 28
4.3.1 Fragebogen 1 - Selbsteinschätzung der ProbandInnen 28
4.3.2 UEQ 28
4.4 Diskussion 28
4.5 Zusammenfassung 28

5 Diskussion & Interpretation **29**

6 Conclusio **31**

Literaturverzeichnis **33**

Abbildungsverzeichnis **37**

Tabellenverzeichnis **39**

Formelverzeichnis **40**

Abkürzungen **42**

Anhang **44**

Tooldokumentation **46**

Eidesstattliche Erklärung **48**

Kurzfassung

Hier ist der Inhalt der Arbeit in komprimierter Form darzustellen.

Länge maximal 1 Seite.

Anmerkung:

Der Leser der Kurzfassung soll verstehen, welche Problemstellung / Fragestellung durch die vorliegende Arbeit bearbeitet wird und welche Erkenntnisse und Ergebnisse vorliegen.

Bitte beachten Sie auch die Richtlinie des Studiengangs zur Erstellung von wissenschaftlichen Arbeiten, vor allem die formalen Anforderungen wie z.B. Zitierweise nach APA-Styleguide in der Form (Autor, Jahr, Seite).

Folgende Seiten beginnen IMMER auf einer rechten, ungeraden Seite:

- Deckblatt
- Vorwort
- Inhaltsverzeichnis
- Kurzfassung
- Abstract
- Neues Kapitel auf Kapitelebene 1. Bitte fügen Sie jeweils entsprechende Leerseiten ein.

Hinweise:

- Die Masterarbeit ist doppelseitig auszudrucken.
- Beinhalten Kapitelüberschriften einen Schrägstrich, so können Sie zwischen den unterschiedlichen Überschriften wählen.

Abstract

Komprimierter Inhalt der Arbeit (Kurzfassung) in Englisch.
Länge maximal 1 Seite.

1 Einleitung

TODO

1.1 Problemstellung

Das Programmierenlernen stellt seit jeher eine erhebliche Hürde in der universitären Ausbildung dar. Wie Medeiros et al. (2019, S. 1) in der systematischen Analyse darlegen, weisen Einführungskurse in die Programmierung weltweit überdurchschnittlich hohe Durchfall- und Abbruchquoten auf. Studierende scheitern oft nicht an der Logik an sich, sondern an der kognitiven Last, die durch die Syntax formaler Sprachen und die Notwendigkeit der abstrakten Problemzerlegung entsteht. Traditionelle Lehrmethoden stoßen hier an ihre Grenzen, da sie oft nicht in der Lage sind, individuelles Feedback in der Frequenz und Tiefe zu liefern, die für das Überwinden dieser initialen Hürden notwendig wäre.

In dieses didaktische Feld treten nun Large Language Models (LLMs) wie ChatGPT, die das Potential haben, die Art und Weise, wie wir Code schreiben und lehren, fundamental zu verändern. Nach Denny et al. (2024b) stehen wir vor einem Paradigmenwechsel. Die Fähigkeit dieser Modelle ausgehend von natürlicher Sprache syntaktisch korrekten Code zu generieren, senkt zudem die Einstiegshürde drastisch (Becker et al., 2023, S. 500-501). Doch diese scheinbare Erleichterung birgt ein massives Risiko für die gute wissenschaftliche Praxis und den Kompetenzerwerb: Wenn Studierende die KI nicht als Werkzeug, sondern als Orakel nutzen, entsteht eine „Illusion of Capability"(Prather et al., 2023, S. 116). Sie produzieren Code, den sie weder verstehen noch warten können, anstatt Problemlösekompetenz zu erwerben.

Ein spezifisches Problem, das in der aktuellen Lage oft übersehen wird, ist die Diskrepanz zwischen der Generierung von isolierten Code-Schnipseln und deren Integration in bestehende, Frameworks. Während LLMs Standardalgorithmen beherrschen, zeigen Studien von Kazemitabaar et al. (2023, S. 7), dass Beginner oft daran scheitern, generierten Code korrekt in vorgegebene Strukturen einzubetten oder Kontextfehler wie z.B. halluzinierte Methodenaufrufe zu erkennen.

Die vorliegende Arbeit adressiert genau dieses Feld, indem sie untersucht, wie sich der Einsatz von ChatGPT auf die Bearbeitung einer komplexeren Aufgabe im Rahmen eines vorgegebenen Python Frameworks („Garden-Simulator“) auswirkt. Anstatt sich auf reine Performance-Metriken zu beschränken, kombiniert diese Studie eine qualitative Code-Analyse basierend auf Funktionalitäts- und Stilapekten mit einer Evaluation der User Experience mittels des User Experience Questionnaires (UEQ). Ziel ist es, evidenzbasiert zu klären, ob LLMs in der Hand von Anfängern tatsächlich zu besserer Software führen und in der Bearbeitung der Aufgabe auch bessere Usabibility-Werte erzielen können, als bei einer klassischen Durchführung ohne LLM.

1.2 Wissenschaftliche Fragestellung

Die in Kapitel 2 durchgeführte Literaturanalyse verdeutlicht im Wesentlichen zwei Aspekte: (1) Es gibt eine Vielzahl an Studien und Paper, die sich mit LLMs und Programmieren allgemein oder im engeren Sinne mit dem Einsatz von LLMs zum

Programmierenlernen beschäftigen. So fokussieren z.B. Finnie-Ansley et al. (2022) oder Kazemitabaar et al. (2023) Performance-Metriken wie Zeit, Korrektheit und Code-Qualität. Der Aspekt der User Experience bei der Interaktion mit einem LLM-Chatbot wird jedoch wenig bis selten betrachtet. Die Arbeiten von Liffiton et al. (2024), Kazemitabaar et al. (2024) und Kazemitabaar et al. (2023) nehmen hierbei eine Sonderstellung ein, da sie Einblicke in die Benutzerfreundlichkeit ihrer Tools bieten. Bei ersterem wurde eine Online-Umfrage durchgeführt, bei zweiterem Interviews sowie wöchentliche Umfragen und bei letzterem wurde Feedback mittels Likert-Skala-Fragen eingeholt. Ein standardisiertes Instrument wie der UEQ kam nicht zum Einsatz. Die Literaturanalyse verdeutlicht (2) auch, dass in Arbeiten, die den UEQ eingesetzen, es primär um die Evaluierung von LLM-Tools geht ohne spezifischen Bezug zur Programmierausbildung.

Daraus ergibt sich folgende Forschungslücke: LLM-Chatbots werden zwar als Hilfe für Programmiereinsteiger bereitgestellt, es fehlt jedoch eine standardisierte Auswertung der User Experience in diesem Bereich. Um diesem Umstand entgegenzuwirken, wird in der vorliegenden Arbeit ein LLM-Chatbot herangezogen – in diesem Fall ChatGPT – mit dessen Unterstützung das Programmieren-lernen bei Einsteigern gefördert werden soll. Dieser Prozess wird nicht nur aus didaktischer Sicht betrachtet werden, sondern auch aus dem Blickwinkel der User Experience.

Die folgenden Forschungsfragen werden im Laufe der Arbeit beantwortet:

RQ1: Inwieweit beeinflusst die Nutzung eines LLMs (ChatGPT) die Leistung von ProgrammieranfängerInnen bei der Ausarbeitung einer Prüfungsaufgabe im Vergleich zu einer Bearbeitung ohne LLM?

RQ2: Wie unterscheiden sich User Experience und subjektive Wahrnehmung der Prüfungssituation zwischen den beiden Versuchsgruppen, gemessen mit dem UEQ sowie offenen Reflexionsfragen?

Gemessen wird RQ1 anhand der Faktoren: Bearbeitungszeit, Anzahl der Programmstarts bis zur Abgabe und Korrektheit der Gesamtlösung. Die Bewertung der Korrektheit erfordert einen detaillierten und einheitlichen Bewertungsraster, anhand dessen der im Versuch produzierte Programmcode evaluiert werden kann.

1.3 Methodischer Ansatz

TODO

1.4 Aufbau der Arbeit

TODO

2 Stand des Wissens / Stand der Technik

Die vorliegende Masterarbeit untersucht den Einfluss von Large Language Models (LLMs) wie ChatGPT auf das Lernen und Prüfen von Programmieranfänger*innen. Der Forschungsstand umfasst mehrere miteinander verknüpfte Bereiche: die besonderen Herausforderungen des Programmierlernens, den Einsatz von LLMs in der Programmierausbildung, deren Rolle in Prüfungs- und Bewertungsszenarien sowie didaktische Faktoren. In diesem Kapitel soll die einschlägige Literatur zusammengeführt werden.

2.1 Herausforderungen beim Programmierenlernen im tertiären Bildungsbereich

Das Erlernen von Programmierkompetenzen als besonders herausfordernd. Zahlreiche Studien zeigen, dass Einführungsveranstaltungen in der Informatik überdurchschnittlich hohe Dropout- und Durchfallsquoten aufweisen. Medeiros et al. (2019, S. 6) identifizieren in ihrer systematischen Literaturübersicht zentrale Einflussfaktoren, die den Erfolg in Einsteigerkursen bestimmen, darunter mathematische Kompetenzen, Problemlösefähigkeiten, Abstraktionsvermögen sowie die Fähigkeit, Fehlermeldungen zu interpretieren und systematisch zu debuggen. Weitere Arbeiten, etwa von Koulouri et al. (2015, S. 2) oder Gomes und Mendes (2010, S. 1), zeigen, dass insbesondere die ersten Lernschritte also das Strukturieren eines Problems und der Übergang von natürlicher Sprache zu formalen Anweisungen sowie der Umgang mit Syntax als besonders frustrierend erlebt werden.

Auf curricularer Ebene bestehen zusätzliche Herausforderungen: Lehrende müssen mit unterschiedlichen Vorkenntnisse umgehen, sinnvolle Aufgabenformate entwickeln und auf eine angemessene Balance zwischen Theorie, Codepraxis und Feedback achten. Vihavainen et al. (2014, S. 7) sowie Xinogalos (2016, S. 7-8) zeigen, dass die Wahl der Programmiersprache und der didaktischen Methode (z. B. pair programming, projektorientiertes Lernen, tutorielle Unterstützung) entscheidend dazu beiträgt, wie gut Studierende grundlegende Konzepte wie Kontrollstrukturen, Datenstrukturen oder Objektorientierung verstehen.

2.2 Die Verwendung von LLMs in der Programmierausbildung

Mit dem Aufkommen von generativen Modellen wie GPT-4, Claude oder Gemini hat sich das Programmierlernen grundlegend verändert. LLMs können Code generieren, erklären, korrigieren und optimieren. Sie können Fehlermeldungen interpretieren, Beispiele erzeugen und Lernende durch ganze Problemlösungsprozesse begleiten. Denny et al. (2024b, S. 1) beschreiben diesen Wandel als einen Paradigmenwechseln, der sowohl Chancen für individualisiertes Lernen als auch Herausforderungen für Didaktik und Prüfungsdesign mit sich bringt.

Eine Reihe von Studien befasst sich mit den unmittelbaren Lerneffekten. Finnie-Ansley et al. (2022, S. 1) sowie Kazemitabaar et al. (2024, S. 1) zeigen, dass Studierende mit LLM-Unterstützung typischerweise schneller zu funktionierenden Lösungen gelangen und weniger Frustrationspunkte erleben – insbesondere in frühen Lernphasen. Ebenso deuten mehrere Arbeiten darauf hin, dass die Fä-

higkeit der Modelle, Fehlermeldungen zu erklären Leinonen et al. (2023, S. 1), erheblich zur Reduktion von Debugging-Zeiten beiträgt.

Gleichzeitig wird in der Forschung intensiv diskutiert, ob die Nutzung solcher Systeme zu oberflächlichen Lernprozessen führt. Becker et al. (2023, S. 1) argumentieren, dass Programmieren möglicherweise nicht mehr so schwer ist wie früher, weil LLMs viele kognitive Hürden übernehmen. Damit stellt sich die Frage, welche Kompetenzen Studierende tatsächlich erwerben, wenn wesentliche Teile der Problemlösung ausgelagert werden.

Einen wichtigen Beitrag für die Praxis liefern zudem spezialisierte Systeme wie CodeHelp von Liffiton et al. (2024, S. 1) oder CodeAid von Kazemitabaar et al. (2024, S. 3), die LLMs mit didaktischen Leitplanken kombinieren. Diese Tools verhindern, dass der User direkt eine Code-Lösung erhält. Sie bieten stattdessen erklärende Unterstützung, wodurch eigenständiges Denken angeregt werden soll. Erste Studien zu CodeAid und ähnlichen Classroom-Assistenten belegen, dass LLMs in realen Kursen bereits als standardisierte Unterstützungswerkzeuge eingesetzt werden können – allerdings nur, wenn klare Regeln zur Nutzung definiert werden.

2.3 Generative KI in Prüfungen und Bewertungsszenarien

Der Einsatz von LLMs stellt traditionelle Prüfungsformate grundlegend infrage. Studien wie die von Pinto et al. (2023, S. 6) zeigen, dass LLMs offene Fragen in vielen Fällen ähnlich wie menschliche Bewertende beurteilen können - allerdings mit typischen Abweichungen: Manche Modelle sind strenger, andere übersehen subtile Fehler. Damit wird die Frage nach Fairness, Reliabilität und Transparenz solcher Bewertungsmethoden zentral.

Parallel dazu diskutieren Denny et al. (2024a, S. 1), Mahon et al. (2024, S. 1) und Prather et al. (2023, S. 108), welche Aufgabenformate noch geeignet sind, individuelle Kompetenzen im LLM-Zeitalter zu prüfen. Aufgaben, die primär auf reiner Codeproduktion beruhen, verlieren an Aussagekraft, da LLMs solche Lösungen oft vollständig generieren. Stärker gefragt sind daher Problemanalyse, Codebewertung, Fehlersuche, Designentscheidungen und das Verfassen erklärender Texte. Diese Komponenten gelten als „LLM-resistent“, weil sie Verständnis und Reflexion voraussetzen und nicht vollständig automatisierbar sind.

2.4 LLM-unterstütztes Programmierenlernen

Mehrere kontrollierte Studien untersuchen, wie sich der Einsatz großer Sprachmodelle auf das Programmierlernen von Anfänger:innen auswirkt. Eine häufig zitierte Arbeit stammt von Kazemitabaar et al. (2023, S. 11): In einem randomisierten Experiment mit 69 Beginner:innen zeigte sich, dass Lernende mit Zugriff auf einen Codex-basierten Code-Generator signifikant häufiger funktionsfähige Programme erzeugten und ihre Aufgaben schneller lösten als die Kontrollgruppe. Gleichzeitig blieben ihre Leistungen bei nachfolgenden manuellen Code-Modifikationen stabil, was ein Hinweis darauf ist, dass der Einsatz von LLMs nicht automatisch zu Abhängigkeit führt.

Eine weitere Analyse von Kazemitabaar et al. (2023, S. 1) liefert Aufschluss über

die Interaktionsmuster von Programmier-Einsteiger*innen bei der Verwendung von OpenAIs Codex. Anhand von Log-Daten identifizierten die Autoren vier distinkte Nutzungsstrategien bei Novizen: „AI Single Prompt“ (Generierung der gesamten Lösung), „AI Step-by-Step“, „Hybrid“ und „Manual“. Die Ergebnisse deuten darauf hin, dass die Art der Interaktion entscheidend für den nachhaltigen Lernerfolg ist: Der „Hybrid“-Ansatz, der eine Mischung aus eigenständigem Schreiben und gezielter KI-Nutzung darstellt, zeigte positive Trends hinsichtlich der Ergebnisse in späteren Retention-Tests. Wohingegen der „AI Single Prompt“-Ansatz eher mit negativen Resultaten korrelierte. Dies unterstreicht, dass der didaktische Mehrwert von LLMs maßgeblich vom Grad der Selbstregulation (Self-Regulation) und der eigenen kognitiven Beteiligung abhängt (Kazemitabaar et al., 2023, S. 12).

Xue et al. (2024, S. 3-6) untersuchten den Einsatz von ChatGPT 3.5 in einem CS1-Java-Kurs. Obwohl ChatGPT syntaktische Fehler reduzieren konnte, zeigten sich keine signifikanten Vorteile in tiefergehenden konzeptuellen Aufgaben. Die Autor:innen betonen, dass LLMs ohne didaktische Einbettung nur begrenzt Lernzuwächse erzeugen. Ein weiteres Experiment von Toba et al. (2024, S. 6-8) demonstriert vor allem Effizienzgewinne: Mit ChatGPT lösten Studierende Programmieraufgaben etwa doppelt so schnell wie ohne. Die Codequalität blieb jedoch vergleichbar. Gleichzeitig weist die Studie auf Risiken wie missbräuchliche Nutzung in Prüfungssituationen hin.

Eine systematische Meta-Analyse von Alanazi et al. (2025, S. 12-15) bestätigt diese Befunde: KI-gestützte Lernsettings verbessern im Durchschnitt Performance und Bearbeitungszeit deutlich, zeigen aber keine konsistenten Effekte auf tiefes Konzeptverständnis.

2.5 User-Experience-Messungen von LLM-Chatbots mittels User Experience Questionnaires (UEQ)

Die Evaluierung von LLM-basierten Chatbots erfordert Messinstrumente, die über reine Leistungsmetriken hinausgehen und die subjektive Wahrnehmung der Interaktion erfassen. Als Standardinstrument hat sich hierbei der User Experience Questionnaire (UEQ) etabliert, da er sowohl pragmatische (zielgerichtete) als auch hedonische (nicht-zielgerichtete) Qualitätsaspekte abdeckt (Schrepp, 2023, S. 2-3).

Aktuelle Studien bestätigen die Eignung dieses Messinstruments für Chatbots: So konnten Mendoza et al. (2022, S. 17) in einer Untersuchung im schulischen Umfeld nachweisen, dass der UEQ verlässliche Aussagen über die Akzeptanz von Lernassistenten liefert, wobei insbesondere die Dimensionen der Effizienz und der Verlässlichkeit (Efficieny und Dependability) als kritische Erfolgsfaktoren identifiziert wurden. Ambacher (2024) hebt in seiner Analyse von LLM-User-Interfaces hervor, dass eine mangelnde Transparenz über die Funktionsweise des Modells oder unerwartetes Systemverhalten direkt die Dimenson Verlässlichkeit negativ beeinflusst. Wird das System als unberechenbar wahrgenommen, sinkt die Akzeptanz, selbst wenn die generierten Antworten technisch hilfreich sein könnten. Demgegenüber verweist Shahu et al. (2025, S. 6) auf das Potential von LLMs, die pragmatische Qualität (Effizienz, Verständlichkeit und Verlässlichkeit) einer Aufgabe oder eines Produkts stark zu steigern. Ihre Arbeit handelt von einem GIS-Assistenten namens MMapGPT", der durch Implementierung natür-

lichsprachlicher Eingaben die Barrieren zur Nutzung dieser komplexer Software senken konnte. Dieser Sachverhalt schlug sich in positiven Werten auf den UEQ-Skalen für Efficiency (Effizienz) und Perspicuity (Verständlichkeit) nieder. Lee et al. (2024, S. 5-6) verwendeten auch den UEQ als Messinstrument für die Evaluierung ihres Chatbots ChatFiveünd konnten in der Dimension Attraktivität sowie dem Bereich Stimulation signifikant höhere Werte erzielen.

Folglich erweist sich der UEQ als das methodisch adäquate Instrument, um nicht nur die funktionale Gebrauchstauglichkeit (Pragmatik), sondern auch die durch generative KI neu entstehenden emotionalen Interaktionsqualitäten (Hedonik) in einem erprobten Messmodell vergleichbar zu erfassen.

2.6 Zusammenfassung

TODO

3 Forschungsmethodik

3.1 Motivation & Zielsetzung

3.2 Studiendesign und Versuchsaufbau

3.3 Erläuterung der Erhebungsinstrumente: persönlicher Fragebogen und UEQ

3.4 Konzeption der Programmieraufgabe: Der „Garden-Simulator“

Nach dem persönlichen Fragebogen und nachdem die Entwicklungsumgebung bereitsteht, soll die Programmieraufgabe von den Studierenden bearbeitet werden. Dieses Kapitel behandelt den Aufbau und Hintergrund des Python-Programms „Garden-Simulator“, die Aufgabenerstellung sowie die erwarteten Lösungen und die Testumgebung.

3.4.1 Der „Garden-Simulator“

Der *Garden-Simulator* ist eine didaktische Python-Applikation, die die Studierenden im Zuge der einstündigen Studie adaptieren sollen. Das Programm wird im folgenden Kapitel anhand der *4+1 View Model*-Methode nach Kruchten (1995) erläutert.

3.4.1.1 Logische Architektur

Kruchten (1995, S. 3) spricht im Bereich der logischen Architektur von der objektorientierten Zerlegung (Dekomposition) der Problemstellung bzw. Anforderung in ihre zentralen Aspekte als Objekte oder Objektklassen. Dabei kommen die Prinzipien der Abstraktion, Kapselung und Vererbung zum Einsatz. Diese Zerlegung dient nicht nur der funktionalen Analyse, sondern auch dazu, gemeinsame Mechanismen und Entwurfselemente in den verschiedenen Teilen des Systems zu identifizieren. Für datenorientierte Anwendungen können auch abweichende Darstellungsformen der logischen Sicht verwendet werden (Kruchten, 1995, S. 3).

Im *Garden-Simulator* wurde aufgrund der didaktischen Ausrichtung und des einstündigen Zeitrahmens mit Absicht auf eine umfangreiche Klassen-Struktur verzichtet. Im Mittelpunkt der logischen Sicht steht, wie in Abb. 3.1 dargestellt, der Garten als Raster aus einzelnen Feldern. Jedes Feld wird durch ein Python-Dictionary mit den Einträgen `state`, `crop`, `water` und `days_as_seed` versehen. Diese enthalten den Zustand, die Pflanzenart, den Wasserstand und die Anzahl der Wachstumstage, wenn ausreichend bewässert wurde. Am Anfang ist das Spielfeld leer, wie in Abb. 3.2 zu sehen ist.

Das `state`-Attribut kann die Werte `empty`, `hole`, `seed` oder `ripe` annehmen. Die Pflanzenart wird in `crop` gespeichert und kann `carrot`, `tomato`, `bean` oder `potato` sein. Auf einem unbepflanzten Feld enthält dieser Eintrag `None`. Die Einträge `water` und `days_as_seed` sind ganzzahlig (integer). `Water` hat einen Maximalwert von 1, was bedeutet, dass ein Feld entweder gegossen ist oder nicht. Wenn das Feld bepflanzt und gegossen ist und ein Tag mit `advance_day` vergeht, steigt der `days_as_seed`-Zähler um 1. Sobald dieser einen Schwellenwert übertrifft, ist der

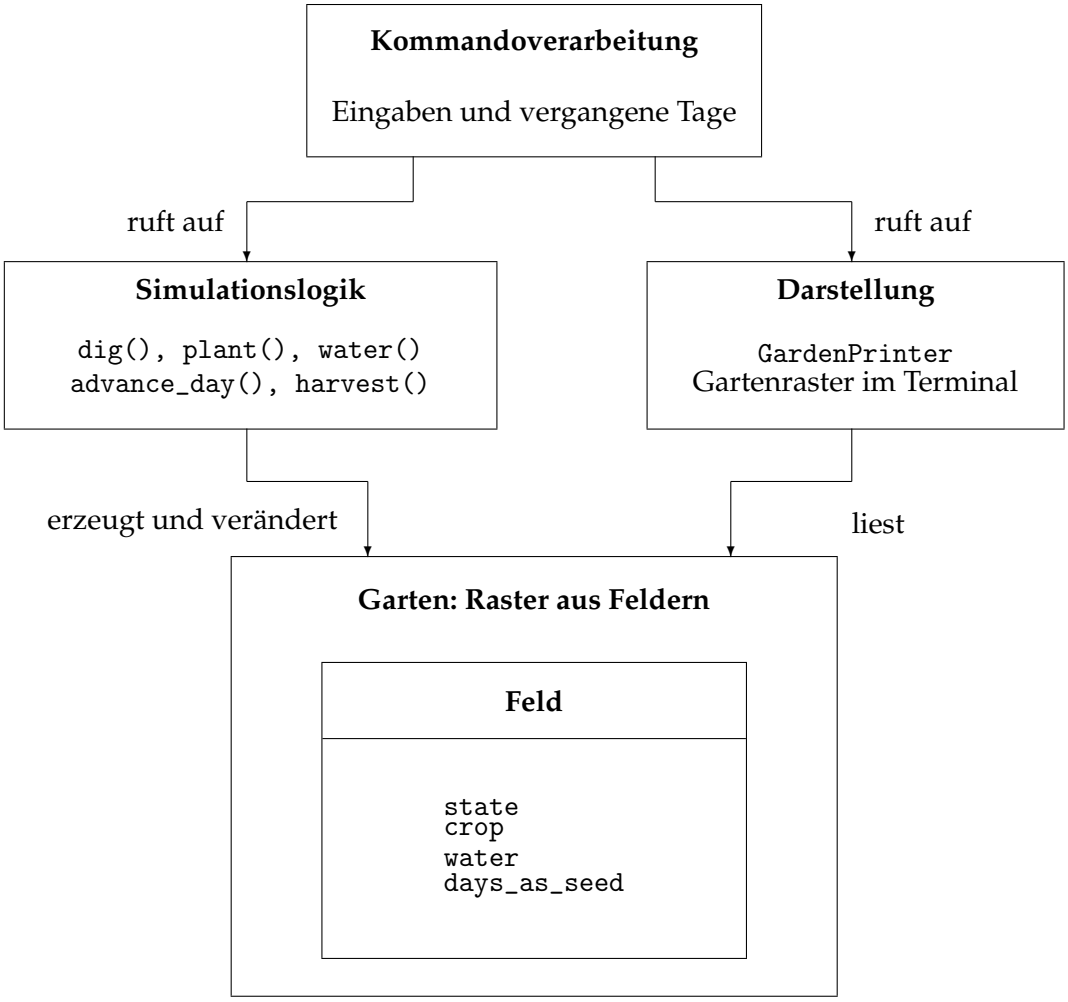


Abbildung 3.1: Logische Architektur des Garden-Simulators. Darstellung nach Kruchten (1995, S. 3).

Status des Feldes `ripe` und die Pflanze kann abgeerntet werden.

Die Simulationslogik umfasst intern die Funktionen `dig()`, `plant()`, `water()` `advance_day()` und `harvest()`, die die jeweiligen Felder des Spielfeldes verändern. Die Befehle folgen bestimmten Regeln, die unten angeführt sind:

- Graben mittels `dig` ist nur auf einem Feld mit dem Zustand `empty` möglich und ändert diesen zu `hole`.
- Pflanzen mittels `plant` ist ausschließlich auf einem Feld mit dem Zustand `hole` erlaubt. Der Zustand wird zu `seed`, die gepflanzte Pflanzenart wird in `crop` eingetragen und `water` sowie `days_as_seed` werden auf null gesetzt.
- Gießen mittels `water` ist nur im Zustand `seed` zulässig und erhöht den Wasserstand höchstens bis zum festgelegten Maximalwert `WATER_MAX` von eins.
- Der `day`-Befehl ruft `advance_day` auf und wirkt auf alle Felder des Gartens. Ein Samen mit einem Wasserstand von mindestens eins erhält einen Wachstumstag und verringert dabei den Wasserstand um -1. Nach 3 Wachstumstagen erhält das Feld den status `ripe`.
- Ernten mittels `harvest` ist nur im Zustand `ripe` möglich. Dabei wird die

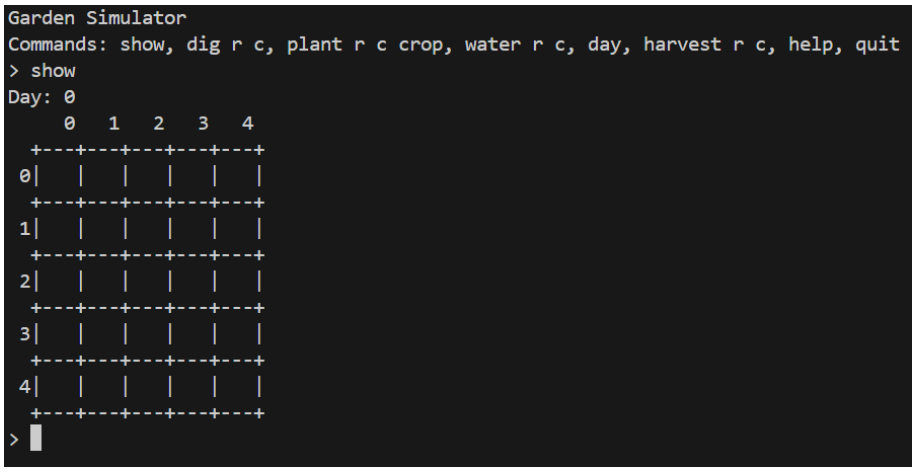


Abbildung 3.2: Startansicht des Garden-Simulators mit ausgeführtem show-Befehl.

Pflanzenart zurückgegeben und das Feld auf `empty` zurückgesetzt. Der Eintrag `crop` wird auf `None` und die beiden Zähler `water` und `days_as_seed` werden auf null gesetzt.

Für die Ausgabe des Spielfeldes im Terminal wird die Klasse `GardenPrinter` verwendet. Dieser Klasse bekommt dieselben Daten wie die Simulationslogik übergeben und bereitet sie als Raster auf, ohne sie zu abzuändern. Die Studierenden müssen diese Klasse nicht adaptieren. Die Kommandoverarbeitung nimmt die Eingaben entgegen und ruft die entsprechenden Funktionen der Simulationslogik oder die Darstellung mittels `show`-Befehls auf. Weiters verwaltet sie die vergangenen globalen Simulationstage in der Variable `current_day`, die bei jedem `day`-Befehl um eins erhöht wird. Dieser Wert ist von den oben erklärtern Wachstumstagen eines einzelnen Feldes zu unterscheiden.

Die genaue Zuordnung der funktionalen Zuständigkeiten von Kommandoverarbeitung, Simulationslogik und Darstellung wird in der Entwicklungs-Architektur in Abschnitt 3.4.1.3 beschrieben.

3.4.1.2 Prozessuale Architektur

Bei der Prozessarchitektur geht es nach Kruchten (1995, S. 4) um die Einordnung der wichtigsten Abstraktionen aus der logischen Sicht in die Prozess-Sicht, also die Frage, auf welchem *thread* (Ausführungsstrang) eine Operation eines Objekts ausgeführt wird. Dabei werden nichtfunktionale Anforderungen wie Leistung und Verfügbarkeit sowie Fragen der Nebenläufigkeit, Synchronisation, Verteilung, Systemintegrität und Fehlertoleranz betrachtet (Kruchten, 1995, S. 4–5).

Ein Prozess wird definiert als eine Gruppierung von Tasks, die eine ausführbare Einheit bilden. Prozesse können z.B. repliziert werden, um die Verarbeitungslast zu verteilen oder die Verfügbarkeit zu erhöhen. Ein Task ist ein unabhängiger und planbarer Ausführungsstrang auf einem Verarbeitungsknoten. Dabei werden eindeutig adressierbare, architektonisch relevante Haupttasks von unterstützenden Nebentasks unterschieden. Letztere übernehmen beispielsweise zyklische Aktivitäten, Pufferung oder die Behandlung von Zeitüberschreitungen. Haupttasks kommunizieren über Mechanismen wie z.B. synchrone und asynchrone nachrichtenbasierte Kommunikation, entfernte Prozeduraufrufe oder Event-Broadcasts

(Kruchten, 1995, S. 4-5).

Im *Garden-Simulator* werden die Kommandoverarbeitung, die Simulationslogik und die Darstellung innerhalb eines einzelnen Python-Prozesses auf einem gemeinsamen Ausführungsstrang ausgeführt wie in Abb. 3.3 dargestellt. Nach Kruchten (1995, S. 13) kann die prozessuale Architekur bei kleinen Projekten mit nur einem Prozess weggelassen werden. Für eine Beschreibung des Programmflusses wurde das Kapitel trotzdem erstellt.

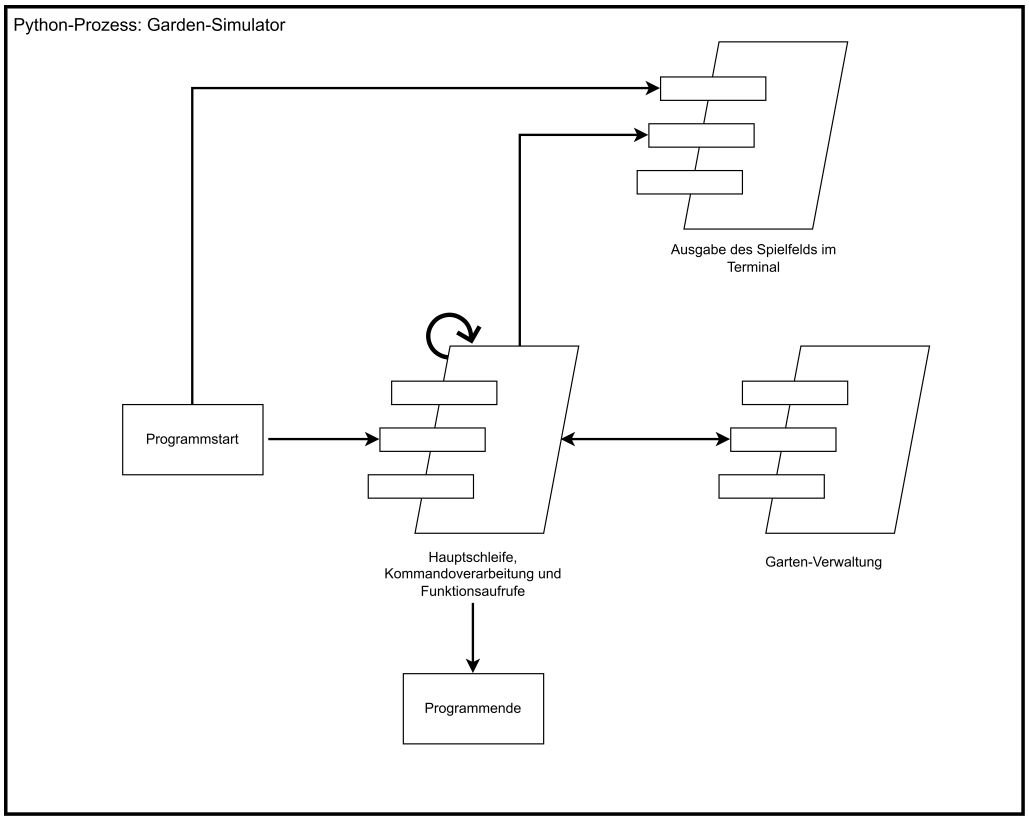


Abbildung 3.3: Prozess-Ansicht des Garden Simulators. Eigene Darstellung nach Kruchten (1995, S. 5-6)

Beim Programmstart werden ein Garten mit fünf Zeilen und fünf Spalten (siehe 3.2), eine Instanz der Klasse `GardenPrinter` und der Tageszähler `current_day` angelegt. Anschließend wartet das Programm in der sogenannten Hauptschleife mittels `input()` auf eine Eingabe. Der eingegebene Befehl wird durch eine Kontrollstruktur ausgewertet und die entsprechende Funktion der Simulationslogik oder die Darstellung aufgerufen. Nach Abschluss der Verarbeitung wird die nächste Eingabe entgegengenommen. Folglich schreitet die Simulation nur durch Benutzereingaben fort. Es gibt auch keine gleichzeitigen oder asynchronen Aufrufe. Mit `quit` oder `exit` wird die Schleife und somit das Programm beendet.

Die in Abb. 3.3 dargestellte Garten-Verwaltung ist mit der Simulationslogik aus der logischen Architektur (siehe Kapitel 3.4.1.1) gleichzusetzen und verändert abhängig vom aufgerufenen Befehl den Zustand des Gartens. Dazu werden die in Abb. 3.1 aufgelisteten Funktionen innerhalb des desselben Python-Prozesses ausgeführt. Die Verarbeitung eines Simulationsbefehls wird abgeschlossen und erst dann nimmt die Hauptschliefe wieder den nächsten Befehl entgegen. Die Ausgabe des Spielfelds im Terminal wird von der Klasse `GardenPrinter` umgesetzt, die

den aktuellen Garten als Raster darstellt. Sie wird wie die Garten-Verwaltung im gleichen Ausführungsstrang ausgeführt.

3.4.1.3 Entwicklungs-Architektur

Die Entwicklungs-Architektur beschreibt nach Kruchten (1995, S. 6) die Organisation der Softwaremodule in ihrer Entwicklungsumgebung. Im Mittelpunkt stehen die modulare Aufteilung sowie deren Import- und Exportbeziehungen. Diese Sicht berücksichtigt insbesondere die einfache Weiterentwicklung, Wiederverwendbarkeit und Verteilung der Entwicklungsaufgaben. Kruchten empfiehlt dazu eine Gliederung in Schichten mit klar definierten Zuständigkeiten, wobei Abhängigkeiten innerhalb derselben oder zu darunterliegenden Schichten verlaufen sollen (Kruchten, 1995, S. 7).



Abbildung 3.4: Übersicht des Projektsverzeichnisses des *Garden-Simulators*.

In Abb. 3.4 ist das Projektverzeichnis des *Garden-Simulator* schematisch abgebildet. Es sind dort die drei Hauptkomponenten `main.py`, `garden.py` und `cli_view.py` gemeinsam vorzufinden. Aufgrund der didaktischen Ausrichtung und der damit erforderlichen Reduktion erfolgt keine weitere Aufteilung in Subsysteme. Wie in Abb. 3.5 dargestellt, bindet `main.py` sowohl die Funktionen aus `garden.py` als auch die Klasse `GardenPrinter` aus `cli_view.py` ein. Es übernimmt dabei die Verarbeitung der Eingaben und verbindet so die Simulationslogik mit der Darstellung. Die beiden eingebundenen Module importieren sich einander nicht und haben keine Importabhängigkeit zu `main.py`. Die Daten aus dem Garten wie z.B. welche Pflanze auf einem Feld platziert ist, werden ihnen als Funktions- bzw. für den `GardenPrinter` als Methodenparameter übergeben.

Das Modul `garden.py` stellt die Funktionen zur Bearbeitung des Gartens bereit und verwendet für die Erntelogik der Planze Tomate die Module `pathlib` und `datetime` aus der Python-Standardbibliothek. Die Datei `cli_view.py` stellt die Klasse `GardenPrinter` zur Verfügung, die die übergebenen Daten des Garten als Raster aufbereitet und im Terminal ausgibt. Diese Aufteilung nach Zuständigkeiten erleichtert es, Änderungen gezielt einem Modul zuzuordnen, was nicht zuletzt ein primärer Faktor für die Planung der Aufgaben war, denn die Studierenden sollten sich z.B. mit der visuellen Darstellung des Rasters nicht beschäftigen. Eine Ausnahme bildet ein `print`-Befehl in `garden.py`, der bei der Ernte erzeugt wird. Dieser ist jedoch von den Studierenden anzupassen und konnte deshalb in `garden.py` bleiben.

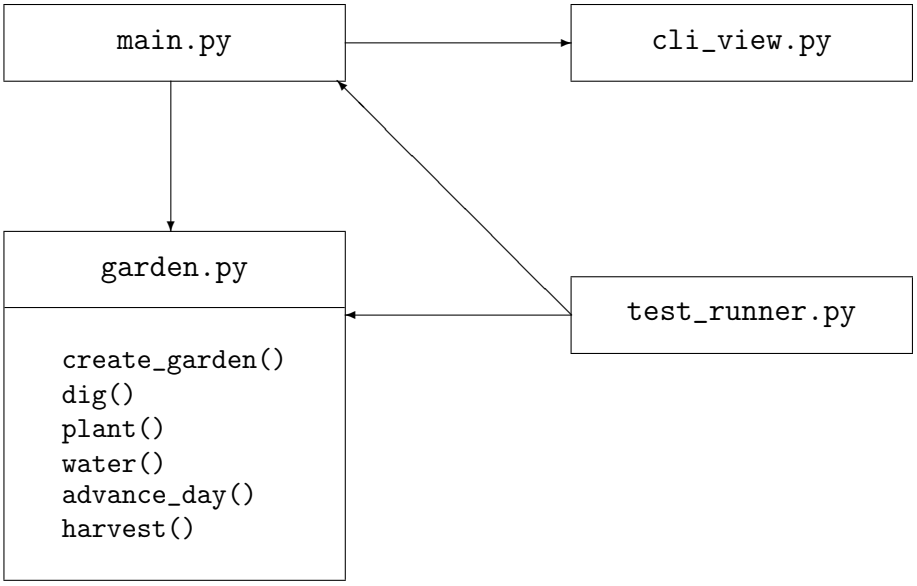


Abbildung 3.5: Entwicklungs-Ansicht des Garden-Simulators. Die Pfeile zeigen vom importierenden zum eingebundenen Modul. Der Testrunner importiert dynamisch über `importlib`. Darstellung nach Kruchten (1995, S. 6–7).

Ergänzt werden die Hauptmodule durch `test_runner.py`, die Dokumentationsdateien `README.md` und `Aufgaben.md` sowie ein `Makefile`. Der Testrunner prüft den Quelltext von `main.py` und lädt die Module `garden.py` und `main.py`. Dieser wird in Abschnitt 3.4.5 detailliert erklärt. Das `Makefile` fasst die Aufrufe zum Starten der Anwendung und der Tests zusammen. Für die Bearbeitung der Programmieraufgabe sind Änderungen in `main.py` und `garden.py` vorgesehen. Die Darstellung und die Tests werden bereitgestellt. Damit unterstützt die Modulaufteilung, wie im letzten Absatz erwähnt, die Eingrenzung der Entwicklungsaufgaben im einstündigen Zeitrahmen der Studie.

3.4.1.4 Physikalische Architektur

Die physikalische Architektur behandelt nach Kruchten (1995, S. 8) die Zuordnung der Software zu den verfügbaren Hardwareknoten, genannt *nodes*. Dabei werden insbesondere Eckpunkte wie Verfügbarkeit, Fehlertoleranz, Leistung und Skalierbarkeit betrachtet. Des Weiteren können unterschiedliche Zuordnungen für Entwicklung, Tests und den tatsächlichen Betrieb beschrieben werden. Für den *Garden-Simulator* werden deshalb zwei Ausführungsvarianten unterschieden: die lokale Ausführung und die Ausführung in GitHub Codespaces während der Prüfungssituation.

In der lokalen Variante werden, wie in Abb. 3.6 a) gezeigt, sämtliche Bestandteile des *Garden-Simulators* einem lokalen Rechner zugeordnet. Die Anwendung wird im Projektverzeichnis mit `python main.py` gestartet. Der dabei ausgeführte Python-Prozess lädt die Module wie in 3.4.1.3 beschrieben. Diese werden nicht als eigenständige Prozesse gestartet. Die Interaktion erfolgt über die Standardeingabe und Standardausgabe des Terminals. Abb. 3.2 zeigt die Ausgabe nach Ausführung des `show`-Befehls. Der Benutzer benötigt für diese Ausführungsvariante keine Netzwerkverbindung.

In der Prüfungssituation wird hingegen GitHub Codespaces mit einem Devcon-

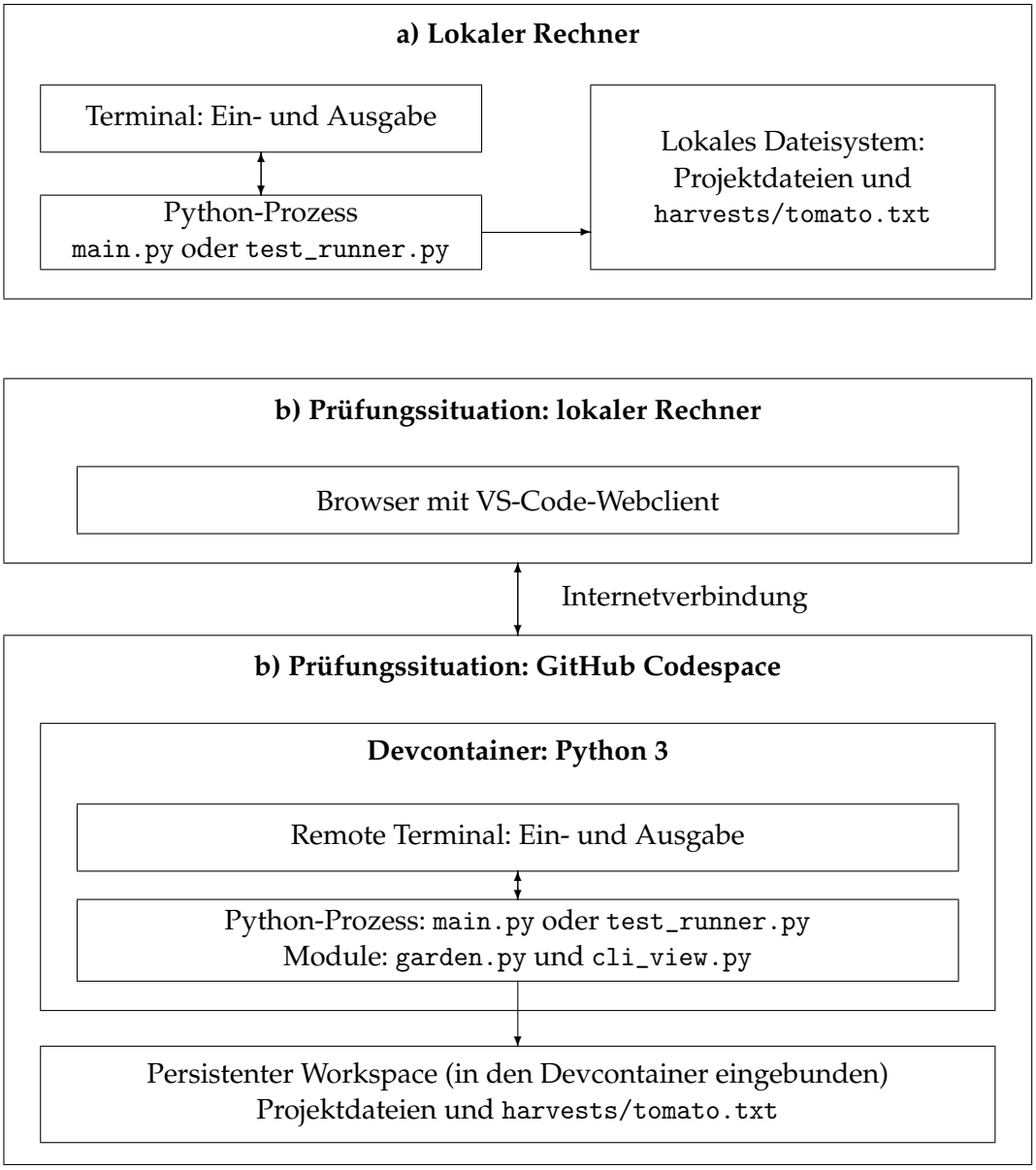


Abbildung 3.6: Physikalische Ansicht des Garden-Simulators: lokale Ausführung (a) und Prüfungssituation mit GitHub Codespaces (b). Eigene Darstellung nach Kruchten (1995, S. 8) und der Codespaces-Infrastruktur (GitHub Docs, 2026).

tainer verwendet. Wie in Abb. 3.6 b) dargestellt, fungiert der lokale Rechner dabei als Zugang zur VS-Code-Webclient Entwicklungsumgebung. Der Devcontainer läuft auf einer von GitHub bereitgestellten virtuellen Maschine (VM) (GitHub Docs, 2026). Die Eingaben und Ausgaben werden über die Netzwerkverbindung zwischen dem lokalen Rechner und dem Codespace übertragen. Die Ausführung der Anwendung und der Tests findet im Devcontainer statt. GitHub Codespaces wurde aus zweierlei Gründen als Lösung für die Prüfungssituation herangezogen. Einerseits weil es durch den Devcontainer für jeden Student/jede Studentinn diesselbe Umgebung bereitstellt. Andererseits kann durch die Konfigurationsdatei die VS-Code Entwicklungsumgebung vorab eingestellt werden und so diverse AI-Features deaktiviert werden, was für den Versuch von zentraler Wichtigkeit war.

Die im letzten Absatz angesprochene Konfigurationsdatei befindet sich unter `.dev`

`container/devcontainer.json` im Projektsverzeichnis (siehe Abb. 3.4). Dort festgelegt sind das Container-Image (`mcr.microsoft.com/devcontainers/python:3`) und die VS-Code-Erweiterungen sowie Editor-Einstellungen. Die voreingestellten Erweiterungen sind `ms-python.python`, `ms-python.vscode-pylance` und ein Markdown-Previewer namens `bierner.markdown-preview-github-styles`. Um eine KI-freie Entwicklungsumgebung garantieren zu können, wurden VS-Code-Einstellungen gesetzt wie z.B. `chat.disableAIFeatures : true`, um KI-Chat-Erweiterungen, oder `editor.inlineSuggest.enabled : false`, um zeilenweise KI-Codevervollständigung auszuschalten. Zudem wurde der Standardformatierer (`editor.defaultFormatter`) auf `NULL` gesetzt sowie `editor.formatOnSave` ausgeschalten, damit diese automatischen Formatierungshilfen die Kategorie „Layout & Formatierung“ des Bewertungsraster (siehe Abschnitt 3.5) nicht beeinflussen.

Das aktuelle Garten-Spielfeld und der Tageszähler, der mit `advance_day` erhöht werden kann, werdenim Arbeitsspeicher des ausführenden Python-Prozesses gespeichert und bei einem Neustart der Anwendung neu angelegt. Dauerhaft gespeichert wird eine Ernte-Log-Datei für Tomaten: Die Funktion `create_tomato_harvest_file` legt, wenn nicht vorhanden, das Verzeichnis `harvests` an und fügt eine Log-Zeile in die Datei `tomato.txt` ein. Beim vorgesehenen Start im Projektverzeichnis entsteht somit die Datei `harvests/tomato.txt` innerhalb des Projekts. Lokal liegt diese Datei auf dem verwendeten Rechner (siehe Abb. 3.6 (a) Lokales Dateisystem), in der Prüfungssituation im Workspace des Codespaces (siehe Abb. 3.6 (b) Persistenter Workspace). GitHub bindet das Repository im Pfad `/workspaces` in den Devcontainer ein und die dort gespeicherten Dateien bleiben beim Stoppen und erneuten Starten des Codespaces erhalten (GitHub Docs, 2026).

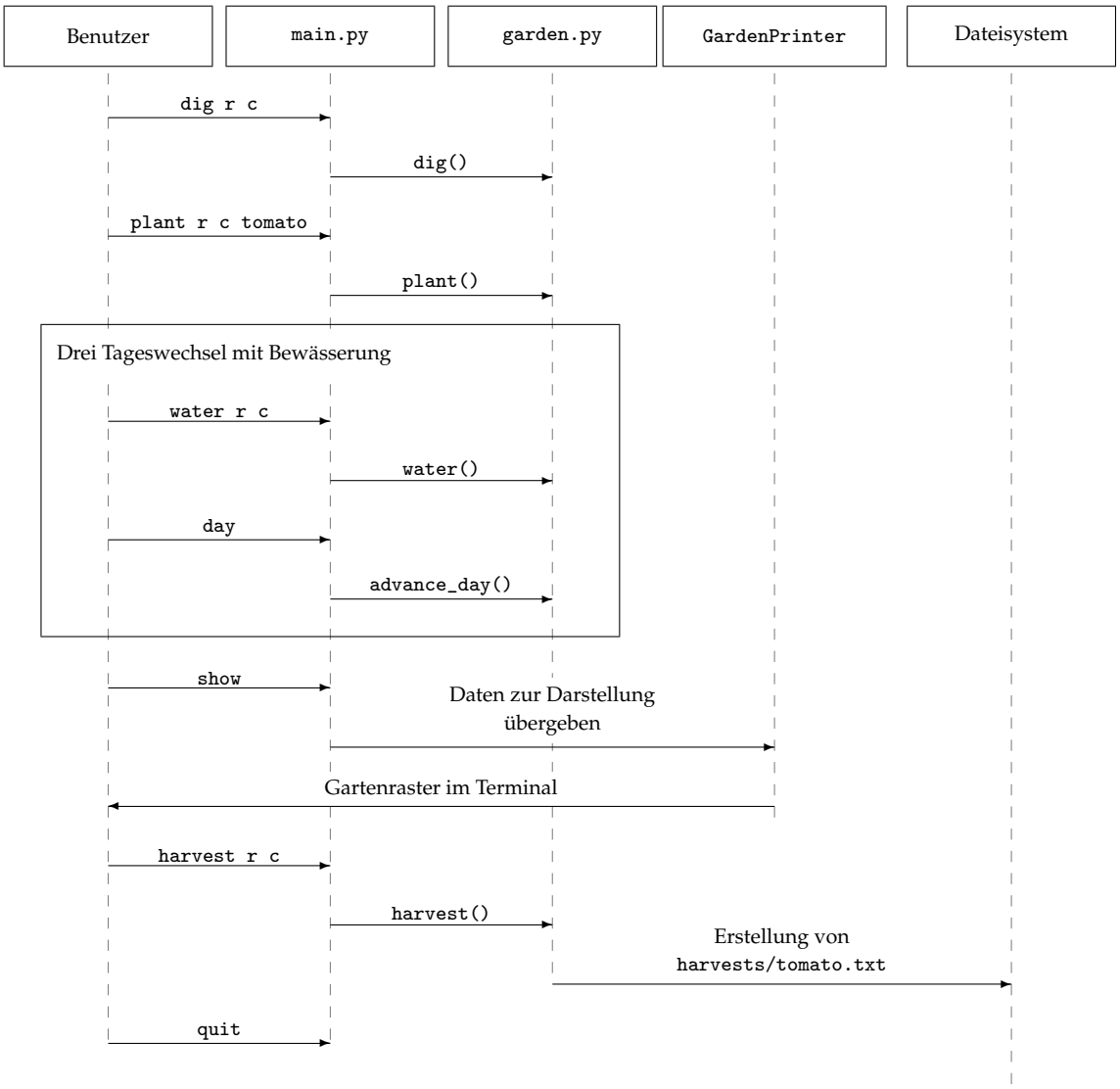
Der Grund für die Erstellung eines Ernte-Logs für Tomaten ist die erste Aufgabe (siehe Abschnitt 3.4.3). Um diese Aufgabe erfolgreich zu absolvieren, müssen die Studierenden das Python-Programm über die Kommandozeile kennenlernen und mindestens einen Ernte-Zyklus für eine Tomate durchspielen. Am Ende des Zyklus wird die `tomato.txt` erstellt, was für den Test-Runner der Indikator dafür ist, dass die erste Aufgabe erfolgreich abgeschlossen wurde. Der Aufruf des Test-Runners mittles `python test_runner.py` startet einen beiden Umgebungen einen eigenen Python-Prozess.

3.4.1.5 Szenario: Anbau und Ernte

Nach Kruchten (1995, S. 9) veranschaulichen Szenarien das Zusammenwirken der Elemente aus den vier Architektursichten anhand einem konkreten Anwendungsfall. Sie beschreiben die Abfolge und die Interaktionen zwischen den beteiligten Elementen. Während der Planungsphase unterstützen sie die Identifikation benötigter Architekturelemente. In einem weiteren Schritt dienen sie der Veranschaulichung und Überprüfung der Architektur und können als Ausgangspunkt für Tests verwendet werden. Für den *Garden-Simulator* wird im Folgenden ein vollständiger Anbau- und Erntezyklus (siehe Abb. 3.7) einer Tomate mit abschließender Testcase-Überprüfung mittels Test-Runners betrachtet. Das Szenario bezieht sich auf das für die Studierenden bereitgestellte Ausgangsprogramm und bildet Aufgabe 1 der Studie ab (siehe Kapitel 3.4.3 für eine Detailansicht der Aufgabenstellung).

Ausgangspunkt ist ein leeres Feld im Garten wie in Abb. 3.2 zu sehen ist. Die Studierenden geben zunächst `dig r c` und anschließend `plant r c tomato` ein,

a) Simulation



b) Anschließende Überprüfung mit dem Test-Runner

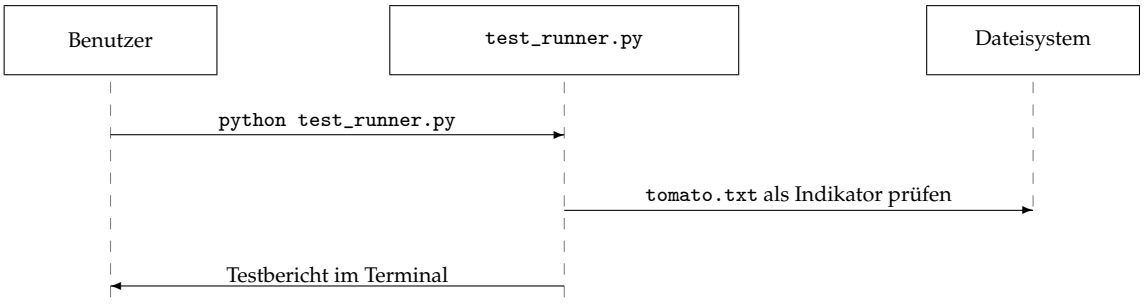


Abbildung 3.7: Interaktionsablauf eines Anbau- und Erntezyluses einer Tomate mit anschließender Test-Runner-Ausführung. Eigene Darstellung nach Kruchten (1995, S. 9–10).

wobei `r` und `c` eine frei wählbare Zeilen- und Spaltenkoordinaten sind. Die Kommandoverarbeitung in `main.py` wertet die Eingaben aus und ruft die entsprechenden Funktionen in `garden.py` auf. Diese ändern den Status des Feldes von `empty` zu `hole` und danach zu `seed`. Beim `plant`-Befehl wird außerdem die Pflanzenart `tomato` eingetragen.

Anschließend führen die Studierenden dreimal nacheinander die Befehle `water r c` und `day` aus. Das Gießen setzt den Wasserstand des Feldes auf eins. Der anschließende Aufruf von `advance_day()` verarbeitet den Tageswechsel für den gesamten Garten. Für das bepflanzte und bewässerte Feld erhöht sich dabei `days_as_seed` um eins. Der Wasserstand wird so wieder auf null herabgesetzt. Nach dem dritten Tageswechsel mit Bewässerung erreicht die Tomate den Zustand `ripe`. Mit `show` können die Studierenden den aktuellen Garten aufrufen. Ein großes `R` markiert die reife Pflanze am ausgegebenen Spielfeld.

Mit `harvest r c` wird anschließend die Ernte ausgelöst. Die Simulationslogik setzt das Feld auf `empty` zurück, entfernt die Pflanzenart und setzt den Wasserstand und die Wachstumstage auf null zurück. Bei der Tomatenernte wird außerdem ein Eintrag in `harvests/tomato.txt` geschrieben. Das Verzeichnis wird, wenn nicht vorhanden, angelegt. Danach beenden die Studierenden den Simulator mit `quit` und starten im selben Ordner mit `python test_runner.py` den Test-Runner. Dieser wird in einem eigenen Python-Prozess ausgeführt und nutzt die vorhandene `tomato.txt` als Indikator für das Bestehen ersten Aufgabe. Der ausgegebene Testbericht enthält auch die Ergebnisse zu den weiteren Aufgaben, die im Ausgangsprogramm noch nicht vollständig erfüllt sind. Die Funktionsweise des Testrunners wird in Abschnitt 3.4.5 erläutert. Es sei an dieser Stelle erwähnt, dass dieser Test von den Studierenden umgangen werden kann, indem sie die Datei im richtigen Verzeichnis selbst erstellen. Um aber an diese Information zu gelangen, erfodert es mehr Verständnis des Programmcodes als in der ersten Aufgabe überhaupt gefordert wird, zumal es bei dieser lediglich um die Interaktion mit der Kommandozeilenschnittstelle geht.

Abb. 3.7 stellt den Anbau- und Erntezyklus einer Tomate und die anschließende Prüfung durch den Test-Runner als zwei aufeinanderfolgende Abläufe dar. Den ersten Ausführungsstrang bilden die squentiellen Aufrufe von Kommandoverarbeitung, Simulationslogik und Darstellung. Die Tomatendatei bleibt nach dem Beenden dieses Prozesses erhalten und steht in weiterer Folge dem Test-Runner zur Verfügung. Lokal liegt sie auf dem verwendeten Rechner im Projektverzeichnis, bei der Ausführung in GitHub Codespaces im dortigen Workspace.

Als Variante dieses Szenarios sei das Ausbleiben der Bewässerung erwähnt. Wird nach dem Pflanzen ein `day`-Befehl ohne vorheriges Gießen ausgeführt, erhöht die Kommandoverarbeitung zwar den globalen Tageszähler `current_day`, das unbewässerte Feld erhält jedoch keinen Wachstumstag. Sein Status als `seed` bleibt erhalten und `days_as_seed` bleibt unverändert. Für die Ernte sind weiterhin drei bewässerte Tageswechsel erforderlich.

3.4.2 README.md als schriftliche Unterstützung

Die `README.md`-Datei dient den Studierenden als Informationsquelle bei der Bearbeitung der Programmieraufgabe. Sie beschreibt das Datenmodell und die Spiel-

regeln, die in Abschnitt 3.4.1.1 erläutert wurden. Weiters enthält sie eine Übersicht der verfügbaren Befehle, eine Erklärung der Status-Symbole im Gartenraster und die Anweisungen zum Starten des Programms. Zwei kurze Tutorials führen durch eine erste Änderung am Code und das Ausführen des Test-Runners. Die Datei soll damit sowohl den Einstieg in die Anwendung unterstützen als auch während der Bearbeitung als Nachschlagewerk dienen.

3.4.3 Erstellung der Aufgaben

Als Programmieraufgabe sollen die Studierenden in der bestehenden Python CLI-Anwendung *Garden-Simulator* bestimmte Use Cases implementieren. Das didaktische Design dieser Unterrichtseinheit basiert auf dem Use-Modify-Create (UMC) Framework, wie es ursprünglich von Lee et al. (2011) definiert wurde. Lytle et al. (2019, S. 398–400) berichten in einer Untersuchung an Schulen über Unterschiede in der wahrgenommenen Schwierigkeit und im Gefühl, den bearbeiteten Code als eigenes Werk anzusehen, die sogenannte *Code-Ownership*. In der Modify-Phase war dieses Gefühl in der UMC-Gruppe stärker ausgeprägt als in der Vergleichsgruppe. Eine erhöhte kognitive Belastung bei erstmaliger Code-Erstellung der Vergleichsgruppe wird als mögliche Erklärung für die subjektiv wahrgenommene Schwierigkeit diskutiert.

Die Programmieraufgabe orientiert sich an den drei Phasen des UMC-Frameworks. In der ersten Phase (Use) geht es um das Kennenlernen und Ausführen des Source Codes. Im Fall des Garden-Simulators entspricht dies Aufgabe 1, in der Befehle in die Kommandozeilenschnittstelle eingegeben und ausgeführt werden. Die zweite Phase (Modify) umfasst die Aufgaben 2 bis 4 und fordert von den Studierenden, den Code iterativ anzupassen. Um diesen gezielt zu verändern, müssen auf Seite der Studierenden zumindest Teile der inneren Logik durchdrungen worden sein.

Die fünfte und abschließende Aufgabe besteht aus einer selbstständigeren Erweiterung des bestehenden Artefakts um eine neue Funktionalität. Was zu implementieren ist, wird in der untenstehenden Aufgabenbeschreibung definiert. Diese Aufgabe wird im vorliegenden Design als Annäherung an die Create-Phase verstanden. Lee et al. (2011, S. 35) beschreiben für diese Phase die Umsetzung eigener Projektideen, betonen aber zugleich die fließenden Übergänge zwischen den Phasen modify und create. Die vorliegende Aufgabe bleibt aus Zeitgründen eine festgelegte Erweiterung, die Umsetzung im Code unterliegt aber keiner genauen Vorgabe.

Die folgenden Aufgaben sind von den Studierenden zu implementieren.

1. Führe den `help` command aus und sehe dir die Liste an Commands an. Gehe einen Harvest-Zyklus für das Crop „tomato“ in der CLI durch. Führe dafür folgende Commands aus: `dig`, `plant`, `water`, `day`, `water`, `day`, `water`, `day`, `harvest`.
2. Für den Command `quit` fehlt die Beschreibung im help-Befehl. Füge
   i. „quit“ als Command-Name hinzu,
   ii. „Exit the simulator“ als Beschreibung und
   iii. „quit“ als Beispiel.

3. Wenn eine Tomate geerntet wird, wird eine Datei names tomato.txt erstellt. Der User soll über das Erstellen der tomato.txt informiert werden. Der Satz „Created file 'tomato.txt' at <current datetime>“ soll in der Konsole ausgegeben werden. Die Aufgabe ist im Code mittels Kommentarblöcken markiert.
   Beispiel:
   `Created file 'tomato.txt' at 18.05.2026 18:36:22.`
4. Wenn keine Tomate geerntet wird, soll bei den anderen Crops in der Konsole der Satz ausgegeben werden: „[Info] The crop <name of the crop> has no file entry.“
   Beispiel:
   `[Info] The crop carrot has no file entry.`
5. Füge ein neues Crop „potato“ hinzu. Man soll es schon nach 2 `water` + `day` Zyklen ernten können. Die Aufgabe ist im Code nicht mittels Kommentarblöcken markiert. Es müssen dazu Änderungen in beiden Dateien main.py und garden.py gemacht werden.

Die Auswahl der den einzelnen Aufgaben zugrundeliegenden Programmierkonzepte wird im Folgenden anhand gesicherter Inhalte aus der Programmierausbildung vollzogen. So identifizieren Tew und Guzdial (2010, S. 99) auf Grundlage einer Lehrbuchanalyse unter anderem Variablen, Zeichenketten, einfache Ein- und Ausgabe, Vergleichsoperatoren und Kontrollstrukturen als gemeinsame konzeptuelle Grundlagen von Einführungskursen. Die Aufgaben des *Garden-Simulators* greifen diese Konzepte auf. Die Zuordnung in Tabelle 3.1 wurde anhand der Aufgabenstellung und der nachfolgenden Musterlösungen (siehe Abschnitt 3.4.4) vorgenommen. Die Auflistung zeigt übersichtsmäßig pro Aufgabe die behandelten Programmier-Konzepte und die Phasenzuordnung der Unterchtsmodelle. Zu erkennen ist außerdem der kontinuierliche Zuwachs an Komplexität der Programmier-Konzepte. So sind es in Aufgabe zwei Zeichenketten, Listen und Tupel und in Aufgabe fünf sind es u.a. Listen, Verzweigungen und Schleifen.

Eine didaktische Orientierung für die Abfolge der Aufgaben bietet neben UMC auch der PRIMM-Ansatz (Predict, Run, Investigate, Modify, Make) von Sentance et al. (2019, S. 13-16). Dort beschreiben wird der Aufbau einer Lerneinheit von der Ausführung und Untersuchung vorhandenen Codes über Änderungen im Code bis zur Erstellung neuer Programme aus einer Problembeschreibung. Die Aufgabenkonzeption des *Garden-Simulator* folgt auch in Teilen diesem Konzept. Die Phasenzuordnung der Aufgaben nach PRIMM-Ansatz ist in 3.1 ebenfalls dargestellt. Dadurch dass PRIMM als umfangreiches Konzept zur Unterrichtsplanung und somit zum Wissenserwerb entwickelt wurde, finden sich in den Aufgaben der Studie keine Predict- und Investigate-Phase. Die Make-Phase findet aufgrund der begrenzten Zeit auch keine Anwendung.

3.4.4 Musterlösung der Aufgaben und Programmierkonzepte

Im Folgenden wird eine mögliche Umsetzung der im vorhergehenden Abschnitt beschriebenen Aufgaben erläutert. Die folgenden Codeausschnitte zeigen die ergänzten bzw. angepassten Stellen in `main.py` und `garden.py`. Dabei wird zwischen den bereits bereitgestellten Programmteilen und den Änderungen unterschieden, die von den Studierenden vorzunehmen sind.

Aufgabe	Inhaltliche Schwerpunkte	UMC-Phase	PRIMM-Phase
1: Anwendung kennenlernen	Befehlsfolge und beobachtbare Zustandsänderungen	Use	Run
2: Hilfe-Ausgabe ergänzen*	Zeichenketten, Listen und Tupel	Modify	Modify
3: Text-Ausgabe ergänzen*	Konsolenausgabe, Variablen und formatierte Ausgabe	Modify	Modify
4: Andere Pflanzen unterscheiden	Bedingte Verzweigung und formatierte Ausgabe	Modify	Modify
5: Kartoffel als neue Pflanze ergänzen	Listen, Variablen, Verzweigungen und Schleifen	Modify+/Create	Modify

Tabelle 3.1: Programmierkonzepte und Phasenzuordnung der Teilaufgaben
*... Kennzeichnung des Änderungsbereichs mittels Kommentaren im Code

Aufgabe 1: Anwendung kennenlernen Für diese Aufgabe ist keine Änderung am Quellcode erforderlich. Listing 3.1 zeigt die Eingaben für einen vollständigen Anbau- und Erntezyklus einer Tomate auf dem Feld mit den Koordinaten `0 0`. Der Ablauf entspricht dem in Abb. 3.7 dargestellten Szenario. Nach dem Vorbild der in Abschnitt 3.4.3 besprochenen *Use-Phase* lernen die Studierenden hier die bestehende Anwendung und die Reihenfolge der für die Ernte erforderlichen Befehle kennen.

```
help
dig 0 0
plant 0 0 tomato
water 0 0
day
water 0 0
day
water 0 0
day
harvest 0 0
```

Listing 3.1: Befehlsfolge zur Bearbeitung von Aufgabe 1.

Aufgabe 2: Hilfe-Ausgabe ergänzen Die Hilfetabelle wird in der Funktion `print_help_table()` aus `main.py` nach Ausführung des `help`-Befehls erzeugt. Die dort erzeugte Liste `rows` enthält für jeden Befehl ein Tupel aus drei Zeichenketten: dem Befehlsnamen, der Beschreibung und einem Beispiel. Für die Erfüllung der zweiten Aufgabe wird dieser Liste der `quit`-Eintrag, gezeigt in Listing 3.2, hinzugefügt. Hierfür müssen die Studierenden den Aufbau der Liste und der Tupel nachvollziehen und einen weiteren Eintrag in derselben Form ergänzen. Die Ausgabe der Tabelle im Terminal ist bereits implementiert. Als Hilfestellung ist der Berech im Code mittels Kommentaren markiert, in den die Änderung für diese Aufgabe einzufügen ist.

```
rows = [
    ...
    ("quit", "Exit the simulator", "quit"),
    ...
]
```

Listing 3.2: Zusätzlicher Eintrag in der Liste rows in main.py.

Aufgabe 3: Text-Ausgabe In der Funktion `create_tomato_harvest_file()` in `garden.py` sind der Zeitstempel und das Schreiben in die Datei bereits implementiert. Die Studierenden ergänzen am Ende der Funktion die in Listing 3.3 gezeigte `print`-Funktion. Die Methode `format()` ersetzt den Platzhalter `{0}` durch den Wert in der Variable `current_datetime`. Die Aufgabe erfordert damit eine Konsolenausgabe und die Verwendung einer Variable innerhalb einer Zeichenkette. Auch bei dieser Aufgabe ist als Hiflestellung der Implementierungsbereich im Code mittles Kommentaren markiert.

```
print("Created file 'tomato.txt' at {0}".format(current_datetime))
```

Listing 3.3: Ergänzte Ausgabe in create_tomato_harvest_file().

Aufgabe 4: Andere Pflanzen unterscheiden Die Funktion `harvest()` prüft bereits, ob eine Tomate geerntet wurde, und ruft in diesem Fall die oben beschriebene Funktion zum Schreiben der Ernte-Nachricht auf (`create_tomato_harvest_file()`). Für alle Pflanzenarten außer Tomaten wird der `else`-Zweig aus Listing 3.4 ergänzt. Die Variable `harvested_crop` enthält die Pflanzenart des Feldes. Ihr Wert wird mittels `format()` in die Meldung im `else`-Zweig eingefügt. Zusätzlich zur formatierten Konsolenausgabe aus Aufgabe 3 ist als Steigerung der Komplexität eine bedingte Verzweigung erforderlich. Zusätzlich wurde bei dieser Aufgabe der Code-Bereich, in welchem die Adaption durchzuführen ist, nicht mehr mit Kommentaren markiert. Dies steigert zudem den Schwierigkeitsgrad der Aufgabe.

```
if harvested_crop == "tomato":
    create_tomato_harvest_file()
else:
    print("[Info] The crop {0} has no file entry.".format(harvested_crop))
```

Listing 3.4: Fallunterscheidung bei der Ernte in harvest().

Aufgabe 5: Kartoffel als neue Pflanze ergänzen Die fünfte Aufgabe erfordert Änderungen in beiden Dateien `main.py` und `garden.py`. In `main.py` wird die Liste `CROPS` um den Eintrag `potato` erweitert, wie es in Listing 3.5 dargestellt ist. Die Kommandoverarbeitung prüft anhand dieser Liste, ob die beim Pflanzen eingegebene Pflanzenart zulässig ist. Die Hilfetabelle, die beim `help`-Befehl ausgegeben wird, verwendet dieselbe Liste und führt dadurch auch die neue Pflanzenart an.

```
CROPS = ["carrot", "tomato", "bean", "potato"]
```

Listing 3.5: Erweiterte Liste der Pflanzenarten in main.py.

In `garden.py` wird für die Kartoffel die zusätzliche Konstante bzw. globale Variable `RIPE_DAYS_FROM_SEED_potato` mit dem Wert zwei angelegt. Die vorhandene Konstante `RIPE_DAYS_FROM_SEED` behält für die übrigen Pflanzenarten den Wert drei. Listing 3.6 zeigt die Initialisierung der beiden Variablen.

```
RIPE_DAYS_FROM_SEED = 3
RIPE_DAYS_FROM_SEED_potato = 2
```

Listing 3.6: Konstanten für die Reifedauer in garden.py.

Die Funktion `advance_day()` wird um eine Fallunterscheidung anhand der Pflanzenart erweitert. Listing 3.7 zeigt die vollständige Funktion der Musterlösung inklusive der Erweiterung der If-Verzweigungen von Zeile sieben bis Zeile 11. In Zeile zehn wird für `potato` der Wachstumszähler mit dem neuen Schwellenwert verglichen, für die übrigen Pflanzenarten mit dem bisherigen Wert (Zeile 15). Erreicht der Zähler den jeweiligen Schwellenwert, erhält das Feld den Zustand `ripe`. Die beiden verschachtelten Schleifen in den Zeilen 5 und 6 sind bereits im Ausgangsprogramm vorhanden. Die Studierenden müssen diese bestehende Struktur nachvollziehen und die Wachstumslogik innerhalb der bestehenden Funktion anpassen. Bei dieser Aufgabe gibt es den größten Gestaltungsfreiraum, zumal auch hier auf Code-Kommentare zur Kennzeichnung der Änderungsbereiche verzichtet wurde.

```
def advance_day(garden):
    if not garden:
        return True

    for row in garden:
        for cell in row:
            if cell["crop"] == "potato":
                    if cell["state"] == "seed" and cell["water"] >= GROWTH_WATER_NEEDED:
                        cell["days_as_seed"] += 1
                    if cell["days_as_seed"] >= RIPE_DAYS_FROM_SEED_potato:
                        cell["state"] = "ripe"
            else:
                if cell["state"] == "seed" and cell["water"] >= GROWTH_WATER_NEEDED:
                    cell["days_as_seed"] += 1
                if cell["days_as_seed"] >= RIPE_DAYS_FROM_SEED:
                    cell["state"] = "ripe"
            cell["water"] -= 1
    return True
```

Listing 3.7: Angepasste Wachstumslogik in advance_day().

Die Reife nach zwei bzw. drei `water`- und `day`-Zyklen bezieht sich auf den vorgesehenen Ablauf ohne zusätzliche unbewässerte Tageswechsel. Als Randnotiz und aus Gründen der Transparenz sei erwähnt, dass die dargestellte Musterlösung eine Eigenheit des Ausgangsprogramms übernimmt. Denn der Wasserstand wird bei jedem Tageswechsel um eins verringert, ohne den Minimalwert auf null zu begrenzen. Nach einem Tag ohne Gießen kann so ein negativer Wasserstand entstehen. In einer aktualisierten Version des *Garden-Simulator* könnte man auf diese Weise das Austrocknen einer Pflanze simulieren.

Wie in diesem Abschnitt zu erkennen war, sind die Änderungen, die von den Studierenden durchzuführen sind, an den Code-Zeilen gemessen, recht gering. Aufgabe 5 bildet hiervon eine Ausnahme. Trotzdem sei erwähnt, dass aus einer geringen Anzahl der zu ändernden Codezeilen nicht sofort auf eine geringe Schwierigkeit für ProgrammieranfängerInnen geschlossen werden kann. Eine Grundlage dafür liefert die Studie von Lister et al. (2004, S. 119–120). Dort wird von der Schwierigkeit der Studierenden berichtet, die Ausgabe kurzer Programme vorherzusagen und passende Ergänzungen für unvollständigen Code auszuwählen. Für die vorliegenden Aufgaben lässt sich daraus übertragen, dass eine

Änderung, die wenige Codezeilen benötigt, für die Studierenden eine relevante Verständnisanforderung enthalten kann. So muss z.B. bei Aufgabe 4 nachvollzogen werden, unter welcher Bedingung die bestehende Ausgabe erfolgt und an welcher Stelle diese einzufügen ist.

Vor allem bei ProgrammiereinsteigerInnen stellt das Verstehen bestehenden Codes oftmals eine Hürde dar, was in den folgenden Papers ähnlich wie bei Lister et al. (2004, S. 119-120) konkretisieren wurde. In der Literaturübersicht von Qian und Lehman (2017, S. 1-5) schreiben die AutorInnen von Verständnsprobleme bei bedingten Verzweigungen und beim Nachvollziehen eines Variablentauschs aus nur drei Codezeilen. Bei der Fehlersuche beobachteten Fitzgerald et al. (2008, S. 105), dass Studierende zwar gefundene Fehler meist beheben konnten, jedoch deren Auffinden Schwierigkeiten bereitete. Eine ergänzende theoretische Einordnung bietet die Übersicht von Duran et al. (2022, S. 40): Die kognitive Belastung hängt davon ab, welche miteinander verknüpften Elemente gleichzeitig verarbeitet werden müssen und welches Vorwissen die Lernenden mitbringen. Aus diesem Grund wurde beim *Garden-Simulator* bzw. in der Vorbereitung der Studie der Schwierigkeitsgrad der Aufgaben tendentiell niedriger gewählt und die Aufgaben auch in der Pilotphase validiert, siehe dafür Abschnitt 4.1.1.

3.4.5 Lokale Tests zur Selbstüberprüfung

Um herauszufinden, ob die jeweilige Teilaufgabe korrekt erledigt wurde, können die Studierenden mittels des Commands `python test_runner.py` lokale Tests ausführen. Ein Report über bestandene und nicht bestandene Testcases, wie in Abb. 3.8 dargestellt, wird im Terminal präsentiert. Die Tests sollen den Programmier-Prozess der Studierenden unterstützen und so einerseits die korrekte Erledigung einer Aufgabe validieren und andererseits als Debugging-Instrument dienen. Letzteres, also das Lösen einzelner Probleme oder Fehler im Code, ist nach Edwards (2004, S. 26) ProgrammiereinsteigerInnen besonders zeitintensiv und resultiert oft in wahllosen Herumprobieren. Idealerweise sollten die Tests von den Studierenden im Rahmen des Test Driven Developments selbst verfasst werden (Edwards, 2004, S. 27–28; Janzen & Saiedian, 2008, S. 532–533). In der vorliegenden Arbeit werden die Tests hingegen aufgrund der begrenzten Bearbeitungszeit bereitgestellt.

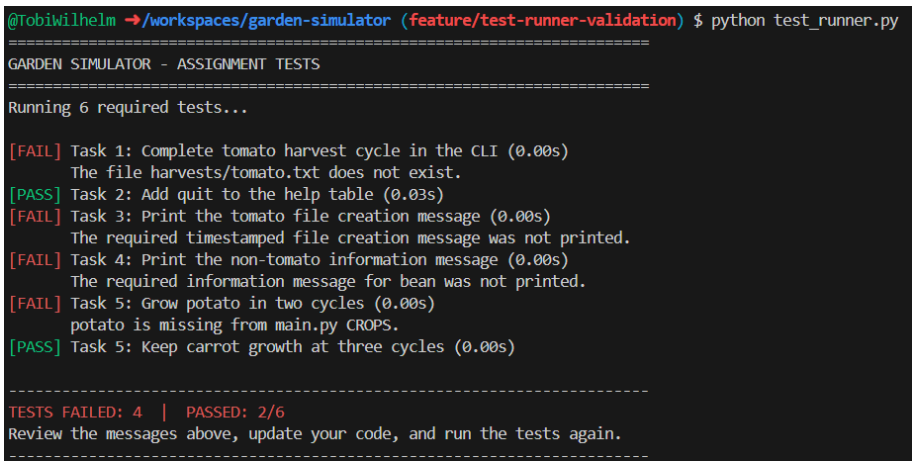


Abbildung 3.8: Ausgabe des lokalen Testrunners bei fehlgeschlagenen und bestandenen Tests

Wie in Abb. 3.8 zu sehen ist, gibt es für jede Teilaufgabe einen eigenen Test. Als Hil-

festellung und um den Testcase zu konkretisieren wird bei jedem fehlgeschlagenen Testcase ein Statustext ausgegeben. So wird z.B. für die nicht erfüllte Teilaufgabe drei ausgegeben: „The required timestamped file creation message was not printed.“ Für Teilaufgabe fünf gibt es jedoch zwei Tests. Denn, wie in der Aufgabenstellung in 3.4.3 erwähnt, müssen hier die benötigten `day`- und `water`-Zyklen bis zur Erntereife für das Crop `potato` adaptiert werden. Es kann hierbei aber passieren, dass die Zyklenanzahl auch für andere Crops verändert wird. Daher wird im letzten Testcase überprüft, ob für ein anderes Crop, in diesem Fall `carrot`, weiterhin die regulären drei `day`- und `water`-Zyklen bis zur Reife erforderlich sind.

3.5 Bewertungsraster zur Evaluierung der Programmieraufgabe

Zur Bewertung der Ausarbeitungen der ProbandInnen wird ein analytisches Bewertungsraster (Rubric) verwendet, das primär auf dem evidenzbasierten Modell von (Stegeman et al., 2016) basiert. Dieses Instrument wurde spezifisch für das Feedback in Einführungskursen der Programmierung (CS1) entwickelt und folgt dem Ansatz eines *Instructional Rubric*. Ziel eines solchen Bewertungsrasters ist es zum einen eine summative Note zu generieren. Zum anderen auch dem Lernenden die Lücke zwischen der aktuellen Leistung und der erwarteten Qualität transparent zu machen. Im Falle der Studie ist der erste Aspekt ausschlaggebend.

Das Raster (Abb. 3.2) bewertet die Code-Qualität anhand von sechs Eigenschaften. Es unterteilt die erbrachte Leistung in vier Erfüllungsgrade. Eine Einordnung in Level 1 bedeutet, dass in dieser Kategorie problematische Teile vorhanden sind. In Level 2 sind die zentralen Kernziele nicht ganz erreicht, es wurde jedoch Code produziert, der keine problematischen Elemente mehr enthält. Ab Level 3 sind die Kernziele erreicht und in Level 4 sogar darüber hinaus. Den Bewertungsstufen werden folgende Punkte zugeordnet: Level 1 entspricht 10 Punkten, Level 2 entspricht 20 Punkten, Level 3 entspricht 30 Punkten und Level 4 entspricht 40 Punkten.

Die einzelnen Leistungsanforderungen sind in Tabelle 3.2 aufgeschlüsselt. Die Kategorien des Rasters wurden aus dem iterativen Design-Prozess für Bewertungsraster von (Stegeman et al., 2016) abgeleitet. Die validierten Kriterien Namen (*names*), Layout & Formatierung (*layout/formatting*), Kontrollfluss (*flow*), Idiomatische Programmierung (*idiom*) und Aufteilung in Funktionen (*decomposition*) werden für den Bewertungsraster der vorliegenden Arbeit adaptiert. So wurden die im Paper ursprünglich für C und Swift entwickelten Deskriptoren für den Kontext der Sprache Python angepasst. Hervorzuheben ist hierbei snake_case für Variablennamen gemäß PEP 8[1].

Zudem wird für die Kategorie Kontrollfluss die Kennzahl Verschachtelung auf Level 1 definiert als mehr als sieben ineinanderfolgende Kontrollstrukturen innerhalb einer Funktion. Auf Level 2 werden Verschachtelungen von mehr als 5 Ebenen eingeordnet. Dabei werden Verzweigungen und Schleifen gemeinsam gezählt. Dieser Schwellenwert orientiert sich am Standardwert für das Attribut *max-nested-blocks* mit dem Wert fünf aus der Pylint-Dokumentation[2].

[1] Link zum Python-Styleguide PEP 8: https://peps.python.org/pep-0008/
[2] Link zur Pylint-Dokumentation: pylint

Kategorie	Gewichtung	Level 1 (Problematisch)	Level 2 (Kernziele nicht erreicht)	Level 3 (Kernziele erreicht)	Level 4 (Kernziele übertroffen)
Funktionalität & Robustheit	0.45	Runtime Errors, grundlegende Anwendungsfälle funktionieren nicht	Ein Teil der Anforderungen ist korrekt implemntiert, bei anderen treten Laufzeitfehler oder falsche Ergebnisse auf oder sind nicht implementiert	Alle Anforderungen werden erfüllt inklusive positver Tests	Zusätzlich zu Level 3 ist der Code robust gegen Randfälle und ungültigen Eingaben
Namen	0.10	Unklare oder irreführende Namen	Namen beschreiben ihren Zweck grunsätzlich zutreffend. Sie weisen aber inkonsistente Schreibweisen auf, sind fehlerhaft geschrieben oder sehr lang	Namen beschreiben ihren Zweck treffend. Konsistentes snake_case, korrekt geschrieben	Zusätzlich zu Level 3 wird im gesamten Programm ein einheitliches Vokabular zur Namensgebung verwendet
Layout & Formatierung	0.10	Erschwerte Lesbarkeit durch eine zu lange Zeilenlänge, falsche Einrückungen oder zurückgelassenen auskommentierten Code	Beabsichtigte Struktur ist grundsätzlich erkennbar. Inkonsistente Formatierung ähnlicher Blöcke	Konsistente Einrückung und Struktur. Zeilenlängen unterstützen die Lesbarkeit	Vollständig PEP 8 konform. Code-Elemente sind einheitlich formatiert und auch über mehrere Dateien konsitent positioniert.
Kontrollfluss	0.15	Verschachtelung > 7 Ebenen, unerreichbarer Code, mehrere Anweisungen pro Zeile	Verschachtelung > 5 Ebenen, viele Sprünge (continue, break oder return), entählt doppelten Code	Einfacher Kontrollfluss, wenige Sonderfälle oder Sprünge, begrenzte Codeduplikation	Zusätzlich zu Level 3 bleibt der häufigste Ausführungspfad auch bei Sonderfällen oder Sprüngen klar erkennbar.
Idiomatische Programmierung	0.10	Kontrollstrukturen werden zweckentfremdet oder irreführend nachgebildet	Unangemessene Wahl der Kontrollstruktur für die jeweilige Aufgabe	Angemessene Wahl der Kontrollstruktur. Verwendung von Bibliotheksfunktionen kann noch begrenzt sein	Zusätzlich zu Level 3 werden Bibliotheksfunktionen und Standard-Datenstrukturen umfassend verwendet.
Aufteilung in Funktionen	0.10	Monolithischer Code, Variablen werden für unterschiedliche Zwecke wiederverwendet lange unübersichtliche Funktionen	Funktionen sind in ihrer Länge begrenzt, vermischen aber unterschiedliche Aufgaben	Funktionen bearbeiten eine überschaubare Menge von Aufgaben mit nachvollziehbarer interner Gliederung	Zusätzlich zu Level 3 bearbeiten Funktionen eine eng abgegrenzte Aufgabe. Geringe Parameterzahl und begrenzte Zahl gemeinsam genutzter Variablen.

Tabelle 3.2: Bewertungsraster V2 nach (Stegeman et al., 2016)

3.5.1 Ergänzung der Dimension „Funktionalität und Robustheit“

Das Modell von (Stegeman et al., 2016) fokussiert sich explizit auf die Code Qualität wie Lesbarkeit, Wartbarkeit und Struktur, die funktionale Korrektheit ist ausklammert. Um jedoch einen umfassendere Bewertung der Abgaben zu gewährleisten, werden Abgaben einer Überprüfung der Laufzeit-Korrektheit unterzogen. Daher wurde das Raster um die Kategorie FFunktionalität & Robustheitërweitert. Diese Kategorie folgt dem Konzept der vier Erfüllungsgrade aus (Stegeman et al., 2016) und wendet sie auf das Laufzeitverhalten an: Level 1 markiert instabilen Code (Abstürze/Runtime Errors), Level 2 funktional falsche Ergebnisse (Logic Errors), Level 3 die korrekte Erfüllung der Spezifikation (Happy Path) und Level 4 eine erhöhte Robustheit gegenüber Randfällen (Edge Cases).

Der Kategorie vorgeschaltet ist Überprüfung der Ausführbarkeit: Code, der aufgrund von Syntaxfehlern nicht vom Interpreter verarbeitet werden kann, fürht dazu dass die Abgabe ein Level abrutscht.

3.6 Planung der Pilotstudie

3.7 Diskussion

3.8 Zusammenfassung

4 Evaluierung der Studie

4.1 Pilotphase der Studie

Bevor die Studierenden mit der Studie konfrontiert werden sollten, wurden die Aufgaben und Abläufe in der Pilotphase erprobt. Die Pilotphase wurde mit insgesamt 3 Personen druchgeführt. Die Auswahl erfolgte aufgrund ihrer Eigenschaften, welche Überschneidungen im Bereich Alter, Studienrichtung oder Vorwissen im Vergleich zur Studiengruppe aufweisen. Im Folgenden sind die Eckdaten der Personen aufgelistet, die die Pilotphase durchlaufen sind:

Person A: 30 Jahre, männlich, Bachelorstudent im Bereich Cloud und IT. Programmierkenntnisse: Gerning. Erfahrung mit Python: Gering

Person B: 28 Jahre, männlich, Bachelorstudent für Lehramt Informatik. Programmierkenntnisse: Mittel. Erfahrung mit Python: Gering

Person C: 26 Jahre, weiblich, Lehrabschluss IT-Technik, Programmierkenntnisse: Gering. Erfahrung mit Python: Gering

Zur Einarbeitung der Verbesserungen wurde ein inkrementelle bzw. agiler Ansatz gewählt. So wurden die drei Personen auch nicht an einem Tag zur Pilotstudie eingeladen. Die Termine wurden mit den Personen nacheinander arrangiert, damit zwischen den Durchführungen genug Zeit vorhanden war, um die Studie anzupassen. Um diese inkrementellen Anpassungen soll es im nun folgenden Kapitel gehen. Es soll auf die Teilaspekte Aufgabenstellung, Entwicklungsumgebung, Quellcode und Bewertungsraster eingeangen weden.

4.1.1 Durchführung und Erkenntnisse der Pilotstudie

Die Termine mit den Personen in der Pilotphase wurden online über Discord abgehalten. Die Versuchspersonen wurden von Anfang an gebeten den Bildschirm freizugeben, damit auch die Schritte vor der Bearbeitung der Aufgaben nachvollzogen weden können.

4.1.1.1 Durchführung mit Person A

Vor dem Termin mit Person A entstand eine rudimentäre Checkliste, die den allgemeinen Ablauf der Studie festhaltet. Die Aufgabenstellung bestand derzeit aus vier Aufgaben mit dem Aufgabenprofil (Aufgabennummer - Klassifikation nach UMC-Framework - Schwierigkeitsgrad): Aufgabe 1 - use - einfach, Aufgabe 2 - modify - einfach, Aufgabe 3 - modify - einfach/mittel, Aufgabe 4 - create/modify - schwer. Person A hat die Aufgaben lokal am persönlichen Windows-11-PC in der Entwicklungsumgebung VS Code bearbeitet. Python war bereits installiert.

Ein zentrales Thema, das während der Durchführung mit Person A auffiel, war die Entwicklungsumgebung. In der Hauptstudie galt das *Bring Your Own Device* - Prinzip und somit musste sichergestellt werden, dass alle TeilnehmerInnen dieselbe Ausgangslage haben. Zwei Lösungsmöglichkeiten wurden im Anschluss diskutiert. Einerseits die Bereitstellung eines vorkonfigurierten Abbilds einer virtuellen Maschine, das mit allen zur Bearbeitung notwendigen Werkzeugen an die Studierenden verteilt werden kann. Andererseits wurde die Verwendung von

GitHub Codespaces in Betracht gezogen, das ebenfalls zentral konfiguriert werden kann, jedoch eine cloudbasierte Lösung darstellt. Ein Codespace wird über den Browser aus einem GitHub-Repository heraus gestartet und man erhält in einem neuen Tab eine VS-Code-Webclient-Oberfläche. Die Entwicklungsumgebung läuft in einem Entwicklungscontainer, genannt DevContainer. Dieser wird über die Datei *devcontainer.json*, die sich im Ordner *.devcontainer* im Repository befindet, konfiguriert. Als Konfigurationsmöglickeiten können unter anderem die benötigten Werkzeuge, Laufzeitumgebungen sowie VS-Code-Erweiterungen und Einstellungen festgelegt werden.

ZIT: https://docs.github.com/en/codespaces/setting-up-your-project-for-codespaces/adding-a-dev-container-configuration/introduction-to-dev-containers

Zusammengefasst fanden in dieser anfänglichen Phase des Pilots die Erstellung der Checkliste sowie die erstmalige Evaluierung des Bewertungsrasters statt. Zudem wurde die zentrale Frage nach der Programmierumgebung aufgeworfen.

4.1.1.2 Durchführung mit Person B

Einen Tag nach Person A wurde mit Person B der Pilot durchgeführt. Die größte Änderung im Vergleich zu

4.1.1.3 Durchführung mit Person C

4.1.2 Adaptionen für die Hauptstudie

4.2 Hauptstudie

4.2.1 Evaluierung

4.3 Auswertung Fragebogen 1 und UEQ (Fragebogen 2)

4.3.1 Fragebogen 1 - Selbsteinschätzung der ProbandInnen

4.3.2 UEQ

4.4 Diskussion

4.5 Zusammenfassung

5 Diskussion & Interpretation

Auf Forschungsfragen eingehen

6 Conclusio

Literaturverzeichnis

Alanazi, M., Soh, B., Samra, H., & Li, A. (2025). The Influence of Artificial Intelligence Tools on Learning Outcomes in Computer Programming: A Systematic Review and Meta-Analysis. https://doi.org/10.3390/computers14050185

Ambacher, J. E. (2024). *Designing a user-friendly and optimized version of a user interface for a large language model (LLM)*. Technische Hochschule Ingolstadt. http://nbn-resolving.de/urn:nbn:de:bvb:573-51556

Becker, B. A., Denny, P., Finnie-Ansley, J., Luxton-Reilly, A., Prather, J., & Santos, E. A. (2023). Programming Is Hard - Or at Least It Used to Be: Educational Opportunities and Challenges of AI Code Generation. *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, 500–506. https://doi.org/10.1145/3545945.3569759

Beispiel, A. (2024). Neue Erkenntnisse in der Forschung. *Journal of Research*, *15*(3), 123–145.

Denny, P., Leinonen, J., Prather, J., Luxton-Reilly, A., Amarouche, T., Becker, B. A., & Reeves, B. N. (2024a). Prompt Problems: A New Programming Exercise for the Generative AI Era. *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*, 296–302. https://doi.org/10.1145/3626252.3630909

Denny, P., Prather, J., Becker, B. A., Finnie-Ansley, J., Hellas, A., Leinonen, J., Luxton-Reilly, A., Reeves, B. N., Santos, E. A., & Sarsa, S. (2024b). Computing Education in the Era of Generative AI. *Commun. ACM*, *67*(2), 56–67. https://doi.org/10.1145/3624720

Duran, R., Zavgorodniaia, A., & Sorva, J. (2022). Cognitive Load Theory in Computing Education Research: A Review. *ACM Transactions on Computing Education (TOCE)*, *22*(4), 40:1–40:27. https://doi.org/10.1145/3483843

Edwards, S. H. (2004). Using software testing to move students from trial-and-error to reflection-in-action. *ACM SIGCSE Bulletin*, *36*(1), 26–30. https://doi.org/10.1145/1028174.971312

Finnie-Ansley, J., Denny, P., Becker, B. A., Luxton-Reilly, A., & Prather, J. (2022). The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. *Proceedings of the 24th Australasian Computing Education Conference*, 10–19. https://doi.org/10.1145/3511861.3511863

Fitzgerald, S., Lewandowski, G., McCauley, R., Murphy, L., Simon, B., Thomas, L., & Zander, C. (2008). Debugging: finding, fixing and flailing, a multi-institutional study of novice debuggers. *Computer Science Education*, *18*(2), 93–116. https://doi.org/10.1080/08993400802114508

GitHub Docs. (2026). *Deep dive into GitHub Codespaces*. Verfügbar 9. Oktober 2026 unter https://docs.github.com/en/codespaces/about-codespaces/deep-dive

Gomes, A. J., & Mendes, A. J. (2010). A study on student performance in first year CS courses. *Proceedings of the Fifteenth Annual Conference on Innovation and Technology in Computer Science Education*, 113–117. https://doi.org/10.1145/1822090.1822123

Janzen, D., & Saiedian, H. (2008). Test-driven learning in early programming courses. *Proceedings of the 39th SIGCSE technical symposium on Computer science education*, 532–536. https://doi.org/10.1145/1352135.1352315

Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3544548.3580919

Kazemitabaar, M., Hou, X., Henley, A., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). How Novices Use LLM-based Code Generators to Solve CS1 Coding Tasks in a Self-Paced Learning Environment. *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*, 1–12. https://doi.org/10.1145/3631802.3631806

Kazemitabaar, M., Ye, R., Wang, X., Henley, A. Z., Denny, P., Craig, M., & Grossman, T. (2024). CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs. *Proceedings of the CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3613904.3642773

Konferenz, P. (2024). Innovative Ansätze in der Technik. *Proceedings of the International Conference on Technology*, 67–89.

Koulouri, T., Lauria, S., & Macredie, R. D. (2015). Teaching Introductory Programming: A Quantitative Evaluation of Different Approaches. *ACM Trans. Comput. Educ.*, *14*(4). https://doi.org/10.1145/2662412

Kruchten, P. (1995). The 4+1 View Model of architecture. *IEEE Software*, *12*(6), 42–50. https://doi.org/10.1109/52.469759

Lee, I., Martin, F., Denner, J., Coulter, B., Allan, W., Erickson, J., Malyn-Smith, J., & Werner, L. (2011). Computational thinking for youth in practice. *ACM Inroads*, *2*(1), 32–37. https://doi.org/10.1145/1929887.1929902

Lee, J., Choi, Y., Song, M., & Park, S. (2024). ChatFive: Enhancing User Experience in Likert Scale Personality Test through Interactive Conversation with LLM Agents. *Proceedings of the 6th ACM Conference on Conversational User Interfaces*. https://doi.org/10.1145/3640794.3665572

Leinonen, J., Hellas, A., Sarsa, S., Reeves, B., Denny, P., Prather, J., & Becker, B. A. (2023). Using Large Language Models to Enhance Programming Error Messages. *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, 563–569. https://doi.org/10.1145/3545945.3569770

Liffiton, M., Sheese, B. E., Savelka, J., & Denny, P. (2024). CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*. https://doi.org/10.1145/3631802.3631830

Lister, R., Adams, E. S., Fitzgerald, S., Fone, W., Hamer, J., Lindholm, M., McCartney, R., Moström, J. E., Sanders, K., Seppälä, O., Simon, B., & Thomas, L. (2004). A multi-national study of reading and tracing skills in novice programmers. *SIGCSE Bull.*, *36*(4), 119–150. https://doi.org/10.1145/1041624.1041673

Lytle, N., Cateté, V., Boulden, D., Dong, Y., Houchins, J., Milliken, A., Isvik, A., Bounajim, D., Wiebe, E., & Barnes, T. (2019). Use, Modify, Create: Comparing Computational Thinking Lesson Progressions for STEM Classes. *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education*, 395–401. https://doi.org/10.1145/3304221.3319786

Mahon, J., Mac Namee, B., & Becker, B. A. (2024). Guidelines for the Evolving Role of Generative AI in Introductory Programming Based on Emerging

Practice. *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*, 10–16. https://doi.org/10.1145/3649217.3653602

Medeiros, R. P., Ramalho, G. L., & Falcão, T. P. (2019). A Systematic Literature Review on Teaching and Learning Introductory Programming in Higher Education. *IEEE Transactions on Education*, *62*(2), 77–90. https://doi.org/10.1109/TE.2018.2864133

Mendoza, S., Sánchez-Adame, L. M., Urquiza-Yllescas, J. F., González-Beltrán, B. A., & Decouchant, D. (2022). A Model to Develop Chatbots for Assisting the Teaching and Learning Process. *Sensors*, *22*(15). https://doi.org/10.3390/s22155532

Mustermann, M. (2023). *Grundlagen der Forschung*. Verlag.

Online, S. (2024). *Digitale Transformation in der Forschung*. Verfügbar 15. Januar 2024 unter https://example.com/digital-transformation

Pinto, G., Cardoso-Pereira, I., Monteiro, D., Lucena, D., Souza, A., & Gama, K. (2023). Large Language Models for Education: Grading Open-Ended Questions Using ChatGPT. *Proceedings of the XXXVII Brazilian Symposium on Software Engineering*, 293–302. https://doi.org/10.1145/3613372.3614197

Prather, J., Denny, P., Leinonen, J., Becker, B. A., Albluwi, I., Craig, M., Keuning, H., Kiesler, N., Kohn, T., Luxton-Reilly, A., MacNeil, S., Petersen, A., Pettit, R., Reeves, B. N., & Savelka, J. (2023). The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*, 108–159. https://doi.org/10.1145/3623762.3633499

Qian, Y., & Lehman, J. (2017). Students' Misconceptions and Other Difficulties in Introductory Programming: A Literature Review. *ACM Transactions on Computing Education (TOCE)*, *18*(1), 1:1–1:24. https://doi.org/10.1145/3077618

Schrepp, M. (2023, 12. September). *User Experience Questionnaire (UEQ) Handbook: All you need to know to apply the UEQ successfully in your projects* [Research report]. Version 11. Verfügbar 2. Dezember 2025 unter https://www.ueq-online.org

Schrepp, M., Hinderks, A., & Thomaschewski, J. (2014). Applying the User Experience Questionnaire (UEQ) in Different Evaluation Scenarios. In A. Marcus (Hrsg.), *Design, User Experience, and Usability. Theories, Methods, and Tools for Designing the User Experience* (S. 383–392). Springer International Publishing. https://doi.org/10.1007/978-3-319-07668-3_37

Sentance, S., Waite, J., & Kallia, M. (2019). Teaching computer programming with PRIMM: a sociocultural perspective. *Computer Science Education*, *29*(2), 136–176. https://doi.org/10.1080/08993408.2019.1608781

Shahu, A., Cipot, P., Asteriou, P., Wintersberger, P., & Michahelles, F. (2025). Integrating AI Assistants to Advance GIS Application Usability. *Proceedings of the 15th International Conference on the Internet of Things*, 1–8. https://doi.org/10.1145/3770501.3770502

Stegeman, M., Barendsen, E., & Smetsers, S. (2016). Designing a rubric for feedback on code quality in programming courses. *Proceedings of the 16th Koli Calling International Conference on Computing Education Research*, 160–164. https://doi.org/10.1145/2999541.2999555

Tew, A. E., & Guzdial, M. (2010). Developing a validated assessment of fundamental CS1 concepts. *Proceedings of the 41st ACM technical symposium on Computer science education*, 97–101. https://doi.org/10.1145/1734263.1734297

Toba, H., Karnalim, O., Johan, M. C., Tada, T., Djajalaksana, Y. M., & Vivaldy, T. (2024). Inappropriate Benefits and Identification of ChatGPT Misuse in Programming Tests: A Controlled Experiment. In M. E. Auer, U. R. Cukierman, E. Vendrell Vidal & E. Tovar Caro (Hrsg.), *Towards a Hybrid, Flexible and Socially Engaged Higher Education* (S. 520–531). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-51979-6_54

Vihavainen, A., Airaksinen, J., & Watson, C. (2014). A systematic review of approaches for teaching introductory programming and their influence on success. *Proceedings of the Tenth Annual Conference on International Computing Education Research*, 19–26. https://doi.org/10.1145/2632320.2632349

Xinogalos, S. (2016). Designing and deploying programming courses: Strategies, tools, difficulties and pedagogy. *Education and Information Technologies*, *21*(3), 559–588. https://doi.org/10.1007/s10639-014-9341-9

Xue, Y., Chen, H., Bai, G. R., Tairas, R., & Huang, Y. (2024). Does ChatGPT Help With Introductory Programming? An Experiment of Students Using ChatGPT in CS1. *2024 IEEE/ACM 46th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, 331–341. https://doi.org/10.1145/3639474.3640076

Abbildungsverzeichnis

3.1 Logische Ansicht des Garden-Simulators 8
3.2 Startansicht des Garden-Simulators mit ausgeführtem show-Befehl. 9
3.3 Prozess-Ansicht des Garden Simulators. Eigene Darstellung nach Kruchten (1995, S. 5-6) 10
3.4 Übersicht des Projektsverzeichnisses des *Garden-Simulators*. 11
3.5 Entwicklungs-Ansicht des Garden-Simulators 12
3.6 Physikalische Ansicht des Garden-Simulators 13
3.7 Szenario: Anbau- und Erntezyluses und anschließende Testprüfung 15
3.8 Ausgabe des lokalen Testrunners bei fehlgeschlagenen und bestandenen Tests 22

Tabellenverzeichnis

3.1 Programmierkonzepte und Phasenzuordnung der Teilaufgaben . . 19
3.2 Bewertungsraster V2 nach (Stegeman et al., 2016) 24

Formelverzeichnis

Abkürzungen

Anhang

Tooldokumentation

Hier sind alle genutzten (KI) Tools zu dokumentieren, die über die im Methodenkapitel eingesetzten Tools hinausgehen.

(KI) Tool	Anwendungsbereich	Betroffener Teil	Anmerkungen

Eidesstattliche Erklärung

„Ich erkläre hiermit an Eides statt, dass ich die vorliegende Arbeit selbständig ohne die Verwendung unerlaubter Hilfsmittel verfasst und keine anderen als die angegebenen Quellen verwendet habe. Alle Stellen, die wörtlich oder sinngemäß den angegebenen Quellen entnommen wurden, sind als solche kenntlich gemacht.

Sofern von der Studiengangsleitung eine Verwendung von Hilfsmitteln (insbesondere IT- und KI-gestützte) vorgesehen ist, erkläre ich, diese in der vorgelegten Arbeit mit dem jeweiligen Produktnamen und einer Beschreibung des genutzten Funktionsumfangs vollständig angeführt zu haben.

Zudem versichere ich, dass ich diese Arbeit gemäß der geltenden Prüfungsordnung der Hochschule für Angewandte Wissenschaften Burgenland sowie den Richtlinien der Österreichischen Agentur für wissenschaftliche Integrität zur guten wissenschaftlichen Praxis verfasst habe. Die Arbeit wurde bisher weder im Inland noch Ausland zur Begutachtung oder Beurteilung vorgelegt und nicht veröffentlicht.“

_____ _____

Ort, Datum Unterschrift